

Quick Reference

Lars Kamm

1 ARM CORTEX-M4F INSTRUCTION-SET

Shifts	Description
omitted	Equivalent to LSL #0
LSL #n	Logical shift left <n> bits. 0 <= <n> <= 31.
LSR #n	Logical shift right <n> bits. 1 <= <n> <= 32.
ASR #n	Arithmetic shift right <n> bits. 1 <= <n> <= 32
ROR #n	Rotate right <n> bits. 1 <= <n> <= 31.
RRX	Rotate right one bit, with extend.

Cycles	Description
P	The number of cycles required for a pipeline refill. Ranges from 1 to 3.
N	The number of registers in the list to be loaded or stored.
W	The number of cycles spent waiting for an event.

Prefix	Description
S	Signed arithmetic modulo 2^8 or 2^{16} . Sets APSR GE flags.
Q	Signed saturating arithmetic.
SH	Signed arithmetic, halving the results. >> 1
U	Unsigned arithmetic modulo 2^8 or 2^{16} . Sets APSR GE flags.
UQ	Unsigned saturating arithmetic.
UH	Unsigned arithmetic, halving the results. >> 1

Suffix	Description
S	If present, the instruction updates the flags. Otherwise, the instruction doesn't update the flags.
R	If R is present, the multiplication is rounded. If R is omitted, the multiplication is truncated.

1.1 32Bit Standard Instruktionen

Group	Description		Syntax			Cycles	Flags									
			Prefix	Operation	Suffix		Registers	N	Z	C	V	GE	3	2	1	0
Data management	Write register Rm or immediate value to Rd. Rd ← Rm Rd ← #imm16	-		{S}{cond}	Rd, Rm Rd, #imm16	1	?	?	?	-	-	-	-	-	-	
	Write a shifted register Rm to Rd. Rd ← shift(Rm)	-	MOV	{S}	Rd, Rm, <shift>	1	?	?	?	-	-	-	-	-	-	
	Write immediate value to top halfword of Rd. Rd _{31:16} ← #imm16	-		T{cond}	Rd, #imm16	1	-	-	-	-	-	-	-	-	-	
	Store immediate or register value in memory. Suffixes: B=Byte: Rt _{7:0} , H=Halfword: Rt _{15:0} , D=Dual: Rt & Rt2, T=Unprivileged, EX=Exclusive	offset ← {Rn +} #imm or shift(Rm) addr ← Rn + offset Mem[addr] ← Rt	-	STR	-, B, H, T, D, BT, HT, EX, EXB, EXH, {cond}	Rt, label Rt, [Rn, #imm]] Rt, [Rn, Rm {, LSL #imm}] Rt, Rt2, [Rn, #imm]]	2	-	-	-	-	-	-	-	-	
	Store multiple registers to consecutive memory locations. addr = Rn for i: Mem[addr] ← Ri addr ← addr + 4	-	STM	{cond}	Rn{!}, list	1+N	-	-	-	-	-	-	-	-	-	
	Load from memory and store to register. Suffixes: B=Byte: Rt _{7:0} , H=Halfword: Rt _{15:0} , D=Dual: Rt & Rt2, S=Signed, T=Unprivileged, EX=Exclusive	offset ← {Rn +} #imm or shift(Rm) addr ← Rn + offset Rt ← Mem[addr]	-	LDR	-, B, H, T, D, BT, HT, SB, SH, SBT, SHT, EX, EXB, EXH	Rt, [Rn {, #imm}] Rt, [Rn, Rm {, LSL #imm}] Rt, Rt2, [Rn, #imm]]	2	-	-	-	-	-	-	-	-	
	Load multiple registers from consecutive memory locations. addr ← Rn for i: Ri ← Mem[addr] addr ← addr + 4	-	LDM	{cond}	Rn{!}, list	1+N	-	-	-	-	-	-	-	-	-	
	Store registers to the stack addr ← SP - 4*BitCount(list) for i: Mem[addr] ← Ri addr ← addr + 4	-	PUSH	{cond}	list	1+N	-	-	-	-	-	-	-	-	-	
	Load registers from the stack addr ← SP + 4*BitCount(list) for i: Ri ← Mem[addr] addr ← addr + 4	-	POP	{cond}	list	1+N	-	-	-	-	-	-	-	-	-	
	Clear record of the executing processor that an address had a request for an exclusive access	-	CLREX	{cond}	-	1	-	-	-	-	-	-	-	-	-	-
Arithmetic	Add a register value to an optionally-shifted register value. Rd ← Rn + shift(Rm)	-	ADD	{S}{cond}	{Rd,} Rn, Rm {, shift}	1	?	?	?	?	-	-	-	-	-	
	Add an immediate value to a register value. Rd ← Rn + #imm	-	ADDW	{cond}	{Rd,} Rn, #imm	1	?	?	?	?	-	-	-	-	-	
	Add an immediate or register value and the carry flag value to a register value. Rd ← Rn + #imm + C or Rd ← Rn + shift(Rm) + C	-	ADC	{S}{cond}	{Rd,} Rn, #imm {Rd,} Rn, Rm {, shift}	1	?	?	?	?	-	-	-	-	-	
	Add an immediate value to the PC, write result to Rd. Rd ← PC + #imm	-	ADR	{cond}	Rd, label	1	-	-	-	-	-	-	-	-	-	
	Subtract an optionally-shifted register value from a register value. Rd ← Rn - shift(Rm)	-	SUB	{S}{cond}	{Rd,} Rn, Rm {, shift}	1	?	?	?	?	-	-	-	-	-	
	Subtract (immediate) subtracts an immediate value from a register value. Rd ← Rn - #imm	-	SUBW	{cond}	{Rd,} Rn, #imm	1	?	?	?	?	-	-	-	-	-	
	Subtract an immediate or register value and the NOT(carry flag) value from a register value. Rd ← Rn - #imm - C or Rd ← Rn - shift(Rm) - C	-	SBC	{S}{cond}	{Rd,} Rn, #imm {Rd,} Rn, Rm {, shift}	1	?	?	?	?	-	-	-	-	-	
	Subtract an immediate or register value from an immediate value. Rd ← ~Rn + #imm or Rd ← ~Rn + shift(Rm)	-	RSB	{S}{cond}	{Rd,} Rn, #imm {Rd,} Rn, Rm {, shift}	1	?	?	?	?	-	-	-	-	-	
	Multiply two register values. The least significant 32 bits of the result are written to Rd. Rd ← Rn * Rm	-	MUL	{S}{cond}	{Rd,} Rn, Rm	1	?	?	-	-	-	-	-	-	-	
	Multiply Rn with Rm, and add Ra. The least significant 32 bits of the result are written to Rd. p1 ← Rn * Rm Rd ← p1 _{31:0} + Ra	-	MLA	{cond}	Rd, Rn, Rm, Ra	2	?	?	-	-	-	-	-	-	-	
	Multiply Rn with Rm, and subtract the least significant 32 bits of the result from Ra. p1 ← Rn * Rm Rd ← Ra - p1 _{31:0}	-	MLS	{cond}	Rd, Rn, Rm, Ra	2	-	-	-	-	-	-	-	-	-	
	Multiply two 32-bit values to produce a 64-bit result. p1 ← Rn * Rm RdLo ← p1 _{31:0} RdHi ← p1 _{63:32}	S, U	MULL	{cond}	RdLo, RdHi, Rn, Rm	1	-	-	-	-	-	-	-	-	-	
	Multiply two 32-bit values to produce a 64-bit value, and accumulate with a 64-bit value. p1 ← Rn * Rm + Int(Rd) RdLo ← p1 _{31:0} RdHi ← p1 _{63:32}	S, U	MLAL	{cond}	RdLo, RdHi, Rn, Rm	1	-	-	-	-	-	-	-	-	-	
	Divide a 32-bit signed integer register value by a 32-bit signed integer register value. Rd ← Rn / Rm	S, U	DIV	{cond}	{Rd,} Rn, Rm	2 to 12	-	-	-	-	-	-	-	-	-	-
	Un/Signed saturation of an optionally shifted value to an un/signed range. #imm = bit position for saturation in the range 0 to 31	S, U	SAT	{cond}	Rd, #imm, Rn {, shift}	1	-	-	-	-	-	-	-	-	-	?



Group	Description	Syntax				Cycles	Flags					
		prefix	operation	suffix	registers		N	Z	C	V	GE	Q
Logic	Bitwise AND of register/immediate or register/ register value. Second register can be shifted.	Rd ← Rn AND #imm or Rd ← Rn AND shift(Rm)	-	AND	{S}{cond} {Rd}, Rn, #imm {Rd}, Rn, Rm {, shift}	1	?	?	?	-	-	-
	Bitwise XOR of register/immediate or register/ register value. Second register can be shifted.	Rd ← Rn XOR #imm or Rd ← Rn XOR shift(Rm)	-	EOR	{S}{cond} {Rd}, Rn, #imm {Rd}, Rn, Rm {, shift}	1	?	?	?	-	-	-
	Bitwise OR of register/immediate or register/ register value. Second register can be shifted.	Rd ← Rn OR #imm or Rd ← Rn OR shift(Rm)	-	ORR	{S}{cond} {Rd}, Rn, #imm {Rd}, Rn, Rm {, shift}	1	?	?	?	-	-	-
	Bitwise OR NOT of register/immediate or register/ register value. Second register can be shifted.	Rd ← Rn OR ~#imm or Rd ← Rn OR ~shift(Rm)	-	ORN	{S}{cond} {Rd}, Rn, #imm {Rd}, Rn, Rm {, shift}	1	?	?	?	-	-	-
	Bitwise AND of register and the complement of immediate or register. Second register can be shifted.	Rd ← Rn AND ~#imm or Rd ← Rn AND ~shift(Rm)	-	BIC	{S}{cond} {Rd}, Rn, #imm {Rd}, Rn, Rm {, shift}	1	?	?	?	-	-	-
	Bitwise inverse of immediate or register. Register can be shifted.	Rd ← ~#imm or Rd ← ~Rn	-	MVN	{S}{cond} Rd, #imm Rd, Rm {, shift}	1	?	?	?	-	-	-
Shift & Rotate	Shift a register value left by a variable or immediate number of bits, shifting in zeros, and write to Rd.	Rd ← Rn << #imm or Rd ← Rn << Rm	-	LSL	{S}{cond} Rd, Rm, #imm Rd, Rm, Rm	1	?	?	?	-	-	-
	Shift a register value right by a variable or immediate number of bits, shifting in zeros, and write to Rd.	Rd ← Rn >> #imm or Rd ← Rn >> Rm	-	LSR	{S}{cond} Rd, Rm, #imm Rd, Rm, Rm	1	?	?	?	-	-	-
	Shift a value right by a variable or immediate number of bits, shifting in copies of the sign bit.	Rd ← signed(Rn) >> #imm or Rd ← signed(Rn) >> Rm	-	ASR	{S}{cond} Rd, Rm, #imm Rd, Rm, Rm	1	?	?	?	-	-	-
	Rotate the value of Rn right by a variable or immediate number of bits. The bits that are rotated off the right end are inserted into the vacated bit positions on the left.		-	ROR	{S}{cond} Rd, Rm, #imm Rd, Rm, Rm	1	?	?	?	-	-	-
	Rotate Right with Extend shifts Rm right by one place, with the carry flag shifted into bit[31].	Rd ← Rm >> 1	-	RXX	{S}{cond} Rd, Rm	1	?	?	?	-	-	-
Test, Compare, Branch	Logical AND operation on register/immediate or register/register value and update the condition flags based on the result, discarding the result. Second register can be shifted.		-	TST	{cond} Rn, #imm Rn, Rm, shift	1	?	?	?	-	-	-
	Perform a XOR operation on register/immediate or register/register value and update the condition flags based on the result, discarding the result. Second register can be shifted.		-	TEQ	{cond} Rn, #imm Rn, Rm, shift	1	?	?	?	-	-	-
	Subtract an immediate or register value from another register and update the condition flags, discard the result.		-	CMP	{cond} Rn, #imm Rn, Rm, shift	1	?	?	?	?	-	-
	Add an immediate or register value from another register and update the condition flags, discard the result.		-	CMN	{cond} Rn, #imm Rn, Rm, shift	1	?	?	?	?	-	-
	Branch conditional or unconditional. Completes in a single cycle if the branch is not taken otherwise add the number of cycles required for a pipeline refill. This ranges from 1 to 3.		-	B	{cond} label	1 or 1+P	-	-	-	-	-	-
	Branch with Link calls a subroutine at a PC-relative address. The assembler calculates the required value of the offset from the PC value of the BL instruction to the label.		-	BL	{cond} label	1 or 1+P	-	-	-	-	-	-
	Branch to an address and instruction set specified by a register.		-	BX	{cond} Rm	1+P	-	-	-	-	-	-
	Calls a subroutine at an address and instruction set specified by a register		-	BLX	{cond} Rm	1+P	-	-	-	-	-	-
	Compare the value in a register with zero. Branch if zero.		-	CBZ	- Rn, label	1 or 1+P	-	-	-	-	-	-
	Compare the value in a register with zero. Branch if non-zero.		-	CBNZ	- Rn, label	1 or 1+P	-	-	-	-	-	-
Bit-Field Processing, Bit Reverse, Extend	A PC-relative forward branch using a table of single byte offsets. A base register provides a pointer to the table, and a second register supplies an index into the table.		-	TBB	{cond} Rn, Rm	2+P	-	-	-	-	-	-
	A PC-relative forward branch using a table of single halfword offsets. A base register provides a pointer to the table, and a second register supplies an index into the table.		-	TBH	{cond} Rn, Rm, LSL #1	2+P	-	-	-	-	-	-
	Count Leading Zeros return in Rm the number of binary zero bits before the first binary one bit in Rm		-	CLZ	{cond} Rd, Rm	1	-	-	-	-	-	-
	Bit Field Clear clears any number of adjacent bits at any position in Rd, without affecting the other bits.	msb = lsb + w - 1 Rd _{msb:lsb} ← Rn _{w-1:0}	-	BFC	{cond} Rd, #lsb, #width	1	-	-	-	-	-	-
	Copy any number of low order bits from Rn into any position in Rd.	msb = lsb + w - 1 Rd _{msb:lsb} ← Rn _{w-1:0}	-	BFI	{cond} Rd, Rn, #lsb, #width	1	-	-	-	-	-	-
	Extract any number of adjacent bits at any position from Rn, sign/zero extends them, and writes to Rd.	msb = lsb + w - 1 Rd ← extend(Rn _{msb:lsb})	S, U	BFX	{cond} Rd, Rn, #lsb, #width	1	-	-	-	-	-	-
	Reverse the byte order in a 32-bit register.	Rd _{31:24} ← Rm _{7:0} Rd _{23:16} ← Rm _{15:8} Rd _{15:8} ← Rm _{23:16} Rd _{7:0} ← Rm _{31:24}	-	REV	{cond} Rd, Rm	1	-	-	-	-	-	-
	Reverse the byte order in each 16-bit halfword of a 32-bit register.	Rd _{31:24} ← Rm _{23:16} Rd _{23:16} ← Rm _{31:24} Rd _{15:8} ← Rm _{7:0} Rd _{7:0} ← Rm _{15:8}	-	REV16	{cond} Rd, Rm	1	-	-	-	-	-	-
	Reverse the byte order in the lower 16-bit halfword of a 32-bit register, and sign extends to 32 bits.	Rd _{31:8} ← signExt(Rm _{7:0}) Rd _{7:0} ← Rm _{15:8}	-	REVSH	{cond} Rd, Rm	1	-	-	-	-	-	-
	Reverse Bits reverses the bit order in a 32-bit register.		-	RBIT	{cond} Rd, Rm	1	-	-	-	-	-	-
State change, Hint, Barriers	Extract an 8/16-bit value from Rm, sign extends it to 32 bits, and write to Rd.	Rd ← signExt(ROR(Rm _{15:0} , #imm))	S, U	XT	B, H Rd, Rm {, ROR #imm}	1	-	-	-	-	-	-
	Generate a call to a system supervisor		-	SVC	{cond} #imm	-	-	-	-	-	-	-
	Move the value from the selected special-purpose register into a general-purpose register.		-	MRS	{cond} Rd, SR	1 or 2	-	-	-	-	-	-
	Move the value of a general-purpose register to the selected special-purpose register.		-	MSR	{cond} SR, Rn	1 or 2	-	-	-	-	-	-
	Breakpoint causes a DebugMonitor exception or a debug halt to occur depending on the configuration of the debug support.		-	BKPT	- #imm	-	-	-	-	-	-	-
	It causes an event to be signaled to all CPUs within the multiprocessor system.		-	SEV	{cond} -	1	-	-	-	-	-	-
	If the Event Register is clear, it suspends execution in the lowest power state available consistent with a fast wakeup, until a reset, exception or other event occurs.		-	WFE	{cond} -	1+W	-	-	-	-	-	-
	It suspends execution, in the lowest power state available consistent with a fast wakeup, until a reset, asynchronous exception or other event occurs.		-	WFI	{cond} -	1+W	-	-	-	-	-	-
	No Operation, does nothing.		-	NOP	{cond} -	1	-	-	-	-	-	-
	Flush the pipeline in the processor, so that all instructions following the ISB are fetched from cache or memory after the instruction has completed.		-	ISB	{cond} -	1+P	-	-	-	-	-	-
	It ensures that all explicit memory accesses that appear in program order before the DMB instruction are observed before any explicit memory accesses that appear in program order after the DMB instruction.		-	DMB	{cond} -	1	-	-	-	-	-	-
	No instruction in program order after this instruction can execute until this instruction completes.		-	DSB	{cond} -	1	-	-	-	-	-	-
	Enable (IE) or disable (ID) the interrupts.		-	CPS	IE / ID flags	1 or 2	-	-	-	-	-	-

1.2 DSP & SIMD Instruktionen

Group	Description		Syntax				Cycles	Flags									
			Prefix	Operation	Suffix	Registers		N	Z	C	V	GE	3	2	1	0	Q
Multiply	Multiply two signed 32-bit values, extract the most significant 32 bits, and add the value to Ra. If R is present, the multiplication is rounded.	res ← (Ra << 32) + Rn * Rm Rd ← res _{63:32}	-	SMMLA	{R}{cond}	Rd, Rn, Rm, Ra	1	-	-	-	-	-	-	-	-	-	-
	Multiply two signed 32-bit values, extract the most significant 32 bits, and subtract the value from Ra. Optionally round the result.	res ← (Ra << 32) - Rn * Rm Rd ← res _{63:32}	-	SMMLS	{R}{cond}	Rd, Rn, Rm, Ra	1	-	-	-	-	-	-	-	-	-	-
	Multiply two signed 32-bit values, extracts the most significant 32 bits of the result. If R is present, the multiplication is rounded.	res ← Rn * Rm Rd ← res _{63:32}	-	SMMUL	{R}{cond}	{Rd,} Rn, Rm	1	-	-	-	-	-	-	-	-	-	-
	Multiply two signed 16-bit quantities, taken from either the bottom or the top half.	x=B:p1 ← Rn _{15:0} , x=T:p1 ← Rn _{31:16} y=B:p2 ← Rm _{15:0} , y=T:p2 ← Rm _{31:16} Rd _{31:0} ← p1 * p2	-	SMULxy	{cond}	{Rd,} Rn, Rm	1	-	-	-	-	-	-	-	-	-	-
	Multiply a signed 32-bit quantity and a signed 16-bit quantity. The 16-bit quantity is taken from the bottom or top half of Rm.	y=B:p2 ← Rm _{15:0} , y=T:p2 ← Rm _{31:16} tmp ← Rn * p2 Rd _{31:0} ← tmp _{47:16}	-	SMULWy	{cond}	{Rd,} Rn, Rm	1	-	-	-	-	-	-	-	-	-	-
	Two signed 16 × 16-bit multiplications and adds the products together.	p1 ← Rn _{15:0} * Rm _{15:0} p2 ← Rn _{31:16} * Rm _{31:16} Rd _{31:0} ← p1 + p2	-	SMUAD	{cond}	{Rd,} Rn, Rm	1	-	-	-	-	-	-	-	-	-	?
	Two signed 16 × 16-bit multiplications and adds the products together. Halfwords of Rm are exchanged.	p1 ← Rn _{15:0} * Rm _{31:16} p2 ← Rn _{31:16} * Rm _{15:0} Rd _{31:0} ← p1 + p2	-	SMUADX	{cond}	{Rd,} Rn, Rm	1	-	-	-	-	-	-	-	-	-	?
	Two signed 16 × 16-bit multiplications and subtracts one product from the other.	p1 ← Rn _{15:0} * Rm _{15:0} p2 ← Rn _{31:16} * Rm _{31:16} Rd _{31:0} ← p1 - p2	-	SMUSD	{cond}	{Rd,} Rn, Rm	1	-	-	-	-	-	-	-	-	-	-
	Two signed 16 × 16-bit multiplications and subtracts one product from the other. Halfwords of Rm are exchanged.	p1 ← Rn _{15:0} * Rm _{31:16} p2 ← Rn _{31:16} * Rm _{15:0} Rd _{31:0} ← p1 + p2	-	SMUSDx	{cond}	{Rd,} Rn, Rm	1	-	-	-	-	-	-	-	-	-	-
	Multiply accumulate, multiply acts on two signed 16-bit quantities. The 32-bit product is added to Ra.	x=B:p1 ← Rn _{15:0} , x=T:p1 ← Rn _{31:16} y=B:p2 ← Rm _{15:0} , y=T:p2 ← Rm _{31:16} Rd _{31:0} ← p1 * p2 + Ra	-	SMLAxy	{cond}	Rd, Rn, Rm, Ra	1	-	-	-	-	-	-	-	-	-	?
	Multiply accumulate, multiply acts on a 32-bit quantity and a 16-bit quantity. The top 32 bits of the 48-bit product are added to Ra.	y=B:p2 ← Rm _{15:0} , y=T:p2 ← Rm _{31:16} tmp ← Rn * p2 + (Ra << 16) Rd _{31:0} ← tmp _{47:16}	-	SMLAWy	{cond}	Rd, Rn, Rm, Ra	1	-	-	-	-	-	-	-	-	-	?
	Two signed 16 × 16-bit multiplications and add the product to Ra.	p1 ← Rn _{15:0} * Rm _{15:0} p2 ← Rn _{31:16} * Rm _{31:16} Rd _{31:0} ← p1 + p2 + Ra	-	SMLAD	{cond}	Rd, Rn, Rm, Ra	1	-	-	-	-	-	-	-	-	-	?
	Two signed 16 × 16-bit multiplications and add the products to Ra. Halfwords of Rm are exchanged.	p1 ← Rn _{15:0} * Rm _{31:16} p2 ← Rn _{31:16} * Rm _{15:0} Rd _{31:0} ← p1 + p2 + Ra	-	SMLADX	{cond}	Rd, Rn, Rm, Ra	1	-	-	-	-	-	-	-	-	-	?
	Two signed 16 × 16-bit multiplications and add the difference of the products to Ra.	p1 ← Rn _{15:0} * Rm _{15:0} p2 ← Rn _{31:16} * Rm _{31:16} Rd _{31:0} ← p1 - p2 + Ra	-	SMLSD	{cond}	Rd, Rn, Rm, Ra	1	-	-	-	-	-	-	-	-	-	?
	Two signed 16 × 16-bit multiplications and add the difference of the products to Ra. Halfwords of Rm are exchanged.	p1 ← Rn _{15:0} * Rm _{31:16} p2 ← Rn _{31:16} * Rm _{15:0} Rd _{31:0} ← p1 - p2 + Ra	-	SMLSDx	{cond}	Rd, Rn, Rm, Ra	1	-	-	-	-	-	-	-	-	-	?
	Multiply two signed 16-bit values to produce a 32-bit value, and accumulate this with a 64-bit value.	x=B:p1 ← Rn _{15:0} , x=T:p1 ← Rn _{31:16} y=B:p2 ← Rm _{15:0} , y=T:p2 ← Rm _{31:16} res ← p1 * p2 + Sint(Rd _{63:0}) RdLo ← res _{31:0} RdHi ← res _{63:32}	-	SMLALxy	{cond}	RdLo, RdHi, Rn, Rm	1	-	-	-	-	-	-	-	-	-	-
	Two signed 16 × 16-bit multiplications and add the products to a 64-bit value.	p1 ← Rn _{15:0} * Rm _{15:0} p2 ← Rn _{31:16} * Rm _{31:16} res ← p1 + p2 + Sint(Rd _{63:0}) RdLo ← res _{31:0} RdHi ← res _{63:32}	-	SMLALD	{cond}	RdLo, RdHi, Rn, Rm	1	-	-	-	-	-	-	-	-	-	-
	Two signed 16 × 16-bit multiplications and add the products to a 64-bit value. Halfwords of Rm are exchanged.	p1 ← Rn _{15:0} * Rm _{31:16} p2 ← Rn _{31:16} * Rm _{15:0} res ← p1 + p2 + Sint(Rd _{63:0}) RdLo ← res _{31:0} RdHi ← res _{63:32}	-	SMLALDX	{cond}	RdLo, RdHi, Rn, Rm	1	-	-	-	-	-	-	-	-	-	-
	Two signed 16 × 16-bit multiplications and add the difference of the products to a 64-bit value.	p1 ← Rn _{15:0} * Rm _{15:0} p2 ← Rn _{31:16} * Rm _{31:16} res ← p1 - p2 + Sint(Rd _{63:0}) RdLo ← res _{31:0} RdHi ← res _{63:32}	-	SMLSLD	{cond}	RdLo, RdHi, Rn, Rm	1	-	-	-	-	-	-	-	-	-	-
	Two signed 16 × 16-bit multiplications and add the difference of the products to a 64-bit value. Halfwords of Rm are exchanged.	p1 ← Rn _{15:0} * Rm _{31:16} p2 ← Rn _{31:16} * Rm _{15:0} res ← p1 - p2 + Sint(Rd _{63:0}) RdLo ← res _{31:0} RdHi ← res _{63:32}	-	SMLSLDX	{cond}	RdLo, RdHi, Rn, Rm	1	-	-	-	-	-	-	-	-	-	-
	Multiple two unsigned 32-bit values to produce a 64-bit value, add two unsigned 32-bit values.	res ← Rn + Rm + RdHi + RdLo RdHi ← res _{63:32} RdLo ← res _{31:0}	-	UMAAL	{cond}	RdLo, RdHi, Rn, Rm	1	-	-	-	-	-	-	-	-	-	-

Group	Description		Syntax				Cycles	Flags					
			Prefix	Operation	Suffix	Registers		N	Z	C	V	GE	Q
Packing and Unpacking	Combine bits[15:0] of Rn with bits[31:16] of the shifted value from Rm.	$Rd_{15:0} \leftarrow Rn_{15:0}$ $Rd_{31:16} \leftarrow Rm_{31:16} \ll \#imm$	-	PKHBT	{cond}	{Rd,} Rn, Rm{, LSL #imm}	1	-	-	-	-	-	-
	Combine bits[31:16] of Rn with bits[15:0] of the shifted value from Rm.	$Rd_{15:0} \leftarrow Rm_{15:0} \gg \#imm$ $Rd_{31:16} \leftarrow Rn_{31:16}$	-	PKHTB	{cond}	{Rd,} Rn, Rm{, ASR #imm}	1	-	-	-	-	-	-
	Signed/Zero-extend two 8-bit values to 16 bits each, add the results to two 16-bit values from another register. #rs can be #8 or #16 or #24.	$Rm_{31:0} \leftarrow Rm_{31:0} \gg \#rs$ $Rd_{15:0} \leftarrow Rn_{15:0} + \text{extend}(Rm_{7:0})$ $Rd_{31:16} \leftarrow Rn_{31:16} + \text{extend}(Rm_{23:16})$	S, U	XTAB16	{cond}	{Rd,} Rn, Rm {, ROR #rs}	1	-	-	-	-	-	-
	Signed/Zero-extend two 8-bit values to 16 bits each. #rs can be #8 or #16 or #24.	$Rm_{31:0} \leftarrow Rm_{31:0} \gg \#rs$ $Rd_{15:0} \leftarrow \text{extend}(Rm_{7:0})$ $Rd_{31:16} \leftarrow \text{extend}(Rm_{23:16})$	S, U	XTB16	{cond}	{Rd,} Rm {, ROR #rs}	1	-	-	-	-	-	-
	Signed/Zero-extend an 16/8-bit value. Rm can be rotated before extracting.	$rot \leftarrow \text{ROR}(Rm, \#rs)$ $Rd_{15:0} \leftarrow \text{extend}(rot_{15:0})$	S, U	XT	B,H{cond}	Rd, Rm {, ROR #rs}	1	-	-	-	-	-	-
	Signed/Zero-extend an 16/8-bit value and adding Rn. Rm can be rotated before extracting.	$rot \leftarrow \text{ROR}(Rm, \#rs)$ $Rd_{15:0} \leftarrow Rn + \text{extend}(rot_{15:0})$	S, U	XTA	B,H{cond}	{Rd,} Rn, Rm {, ROR #rs}	1	-	-	-	-	-	-
	Select Bytes from either Rn or Rm according to the values of the GE flags.	$Rd_{7:0} \leftarrow \text{if } GE[0] \text{ } Rn_{7:0} \text{ else } Rm_{7:0}$ $Rd_{15:8} \leftarrow \text{if } GE[1] \text{ } Rn_{15:8} \text{ else } Rm_{15:8}$ $Rd_{23:16} \leftarrow \text{if } GE[2] \text{ } Rn_{23:16} \text{ else } Rm_{23:16}$ $Rd_{31:24} \leftarrow \text{if } GE[3] \text{ } Rn_{31:24} \text{ else } Rm_{31:24}$	-	SEL	{cond}	{Rd,} Rn, Rm	1	-	-	-	-	-	-
Sat	Saturate halfwords in Rn to the signed/unsigned range specified by the bit position in #imm	#imm = bit position for saturation in the range 1 to 16	S, U	SAT16	{cond}	Rd, #imm, Rn	1	-	-	-	-	-	?
Miscellaneous Data Processing	Unsigned Sum of Absolute Differences. Four unsigned 8-bit subtractions, and adds the absolute values of the differences together.	$Rd \leftarrow Rn_{7:0} - Rm_{7:0} +$ $ Rn_{15:8} - Rm_{15:8} +$ $ Rn_{23:16} - Rm_{23:16} +$ $ Rn_{31:24} - Rm_{31:24} $	-	USAD8	{cond}	{Rd,} Rn, Rm	1	-	-	-	-	-	-
	Unsigned Sum of Absolute Differences and Accumulate. Four unsigned 8-bit subtractions, and adds the absolute values of the differences to a 32-bit accumulate operand.	$Rd \leftarrow Rn_{7:0} - Rm_{7:0} +$ $ Rn_{15:8} - Rm_{15:8} +$ $ Rn_{23:16} - Rm_{23:16} +$ $ Rn_{31:24} - Rm_{31:24} + Ra$	-	USADA8	{cond}	Rd, Rn, Rm, Ra	1	-	-	-	-	-	-
Addition and Subtraction	Add each byte of the second operand to the corresponding byte of the first operand to form the corresponding byte of the result.	$Rd_{7:0} \leftarrow Rn_{7:0} + Rm_{7:0}$ $Rd_{15:8} \leftarrow Rn_{15:8} + Rm_{15:8}$ $Rd_{23:16} \leftarrow Rn_{23:16} + Rm_{23:16}$ $Rd_{31:24} \leftarrow Rn_{31:24} + Rm_{31:24}$	S, SH U, UH Q, UQ	ADD8	{cond}	{Rd,} Rn, Rm	1	-	-	-	-	-	-
	Add top halfwords of Rn to top halfword of Rm, same for bottom halfwords.	$Rd_{15:0} \leftarrow Rn_{15:0} + Rm_{15:0}$ $Rd_{31:16} \leftarrow Rn_{31:16} + Rm_{31:16}$	S, SH U, UH Q, UQ	ADD16	{cond}	{Rd,} Rn, Rm	1	-	-	-	-	-	-
	Add two registers, saturate the result to the 32-bit signed integer range.	$Rd \leftarrow \text{SignSat}(Rm + Rn)$	-	QADD	{cond}	{Rd,} Rm, Rn	1	-	-	-	-	-	-
	Add a doubled register value to another register value. Both operations are saturated.	$d \leftarrow \text{SignSat}(2 * Rn)$ $Rd \leftarrow \text{SignSat}(Rm + d)$	-	QDADD	{cond}	{Rd,} Rm, Rn	1	-	-	-	-	-	-
	Subtract each byte of the second operand to the corresponding byte of the first operand to form the corresponding byte of the result.	$Rd_{7:0} \leftarrow Rn_{7:0} - Rm_{7:0}$ $Rd_{15:8} \leftarrow Rn_{15:8} - Rm_{15:8}$ $Rd_{23:16} \leftarrow Rn_{23:16} - Rm_{23:16}$ $Rd_{31:24} \leftarrow Rn_{31:24} - Rm_{31:24}$	S, SH U, UH Q, UQ	SUB8	{cond}	{Rd,} Rn, Rm	1	-	-	-	-	-	-
	Subtract top halfwords of Rn from top halfword of Rm, same for bottom halfwords.	$Rd_{15:0} \leftarrow Rn_{15:0} - Rm_{15:0}$ $Rd_{31:16} \leftarrow Rn_{31:16} - Rm_{31:16}$	S, SH U, UH Q, UQ	SUB16	{cond}	{Rd,} Rn, Rm	1	-	-	-	-	-	-
	Subtract Rn from Rm, saturates the result to the 32-bit signed integer range.	$Rd \leftarrow \text{SignSat}(Rm - Rn)$	-	QSUB	{cond}	{Rd,} Rm, Rn	1	-	-	-	-	-	-
	Subtract 2 * Rn from Rm, saturates the result to the 32-bit signed integer range.	$d \leftarrow \text{SignSat}(2 * Rn)$ $Rd \leftarrow \text{SignSat}(Rm - d)$	-	QDSUB	{cond}	{Rd,} Rm, Rn	1	-	-	-	-	-	-
	Exchange halfwords of Rm, and then add top halfwords and subtract bottom halfwords.	$Rd_{15:0} \leftarrow Rn_{15:0} - Rm_{31:16}$ $Rd_{31:16} \leftarrow Rn_{31:16} + Rm_{15:0}$	S, SH U, UH Q, UQ	ASX	{cond}	{Rd,} Rn, Rm	1	-	-	-	-	-	-
	Exchange halfwords of Rm, and then subtract top halfwords and add bottom halfwords.	$Rd_{15:0} \leftarrow Rn_{15:0} + Rm_{31:16}$ $Rd_{31:16} \leftarrow Rn_{31:16} - Rm_{15:0}$	S, SH U, UH Q, UQ	SAX	{cond}	{Rd,} Rn, Rm	1	-	-	-	-	-	-

1.3 IT Instruktionen

Nr. of conditional instructions	IT block x, y, z = T (true) E (else)	Examples
1	IT <cond> instr1<cond>	IT EQ ADDEQ R0,R0,R1
2	ITx <cond> instr1<cond> instr2<cond or ~cond>	ITE GE ADDGE R0,R0,R1 ADDLT R0,R0,R3
3	ITxy <cond> instr1<cond> instr2<cond or ~cond> instr3<cond or ~cond>	ITET LS ADDLS R0,R0,R1 ADDHI R0,R0,R3 ADDLS R2,R4,#1
4	ITxyz <cond> instr1<cond> instr2<cond or ~cond> instr3<cond or ~cond> instr4<cond or ~cond>	ITETT NE ADDNE R0,R0,R1 ADDEQ R0,R0,R3 ADDNE R2,R4,#1 MOVNE R5, R3

Condition codes	Meaning Integer	Meaning Float	Condition flags
EQ	Equal	Equal	Z == 1
NE	Not equal	Not equal or unordered	Z == 0
CS	Carry set	Greater than, equal or unordered	C == 1
CC	Carry clear	Less than	C == 0
MI	Minus or negative result	Less than	N == 1
PL	Positive or zero result	Greater than, equal or unordered	N == 0
VS	Overflow	Unordered	V == 1
VC	No overflow	Not unordered	V == 0
HI	Unsigned higher	Greater than or unordered	C == 1 and Z == 0
LS	Unsigned lower or same	Less than or equal	C == 0 or Z == 1
GE	Signed greater than or equal	Greater than or equal	N == V
LT	Signed less than	Less than or unordered	N != V
GT	Signed greater than	Greater than	Z == 0 and N == V
LE	Signed less than or equal	Less than, equal or unordered	Z == 1 or N != V
AL	Always (unconditional)	Always (unconditional)	Any

1.4 Float Instruktionen

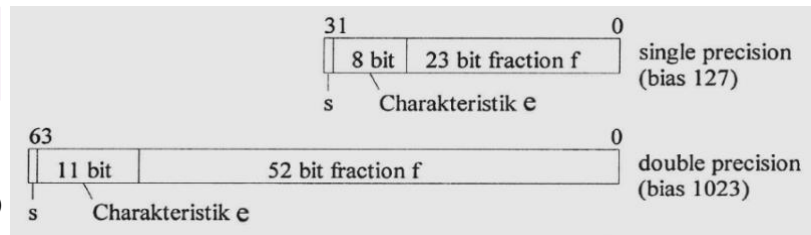
Group	Description	Syntax					Cycles	Flags									
		Prefix	Operation	Suffix	Precision	Register		N	Z	C	V	GE	3	2	1	0	Q
Math operations	Add two single-precision registers and place result in the destination floating-point register	Sd ← Sn + Sm	V	ADD	{cond}	.F32	{Sd,} Sn, Sm	1	-	-	-	-	-	-	-	-	-
	Subtract Sm from Sn and place result in the destination floating-point register	Sd ← Sn - Sm	V	SUB	{cond}	.F32	{Sd,} Sn, Sm	1	-	-	-	-	-	-	-	-	-
	Multiply Sn with Sm, and places the (negated) result in Sd.	V: Sd ← Sn * Sm VN: Sd ← -(Sn * Sm)	V, VN	MUL	{cond}	.F32	Sd, Sn, Sm	1	-	-	-	-	-	-	-	-	-
	Multiply Sn with Sm, and add (the negation) into (negated) Sd.	V: Sd ← Sd + Sn * Sm VN: Sd ← -Sd - Sn * Sm	V, VN	MLA	{cond}	.F32	Sd, Sn, Sm	3	-	-	-	-	-	-	-	-	-
	Multiply Sn with Sm, and subtract the (negated) product from (negated) Sd.	V: Sd ← Sd - Sn * Sm VN: Sd ← -Sd + Sn * Sm	V, VN	MLS	{cond}	.F32	Sd, Sn, Sm	3	-	-	-	-	-	-	-	-	-
	Multiply Sn with Sm, and add (the negation) into (negated) Sd. Product isn't rounded before addition.	VF: Sd ← Sd + Sn * Sm VFN: Sd ← -Sd - Sn * Sm	VF, VFN	MA	AL	.F32	Sd, Sn, Sm	3	-	-	-	-	-	-	-	-	-
	Multiply Sn with Sm, and subtract the (negated) product from (negated) Sd. Product isn't rounded before addition.	VF: Sd ← Sd - Sn * Sm VFN: Sd ← -Sd + Sn * Sm	VF, VFN	MS	AL	.F32	Sd, Sn, Sm	3	-	-	-	-	-	-	-	-	-
	Divide Sn by Sm and write to Sd.	Sd ← Sn / Sm	V	DIV	{cond}	.F32	{Sd,} Sn, Sm	14	-	-	-	-	-	-	-	-	-
	Take the absolute value of a single-precision register	Sd ← Sm	V	ABS	{cond}	.F32	Sd, Sm	1	-	-	-	-	-	-	-	-	-
	Inverts the sign bit of a single-precision register		V	NEG	{cond}	.F32	Sd, Sm	1	-	-	-	-	-	-	-	-	-
	Calculates the square root of a floating-point register	Sd ← sqrt(Sm)	V	SQRT	{cond}	.F32	Sd, Sm	14	-	-	-	-	-	-	-	-	-
Register operations	Compare two registers, writes the result to the FPSCR flags.		V	CMP	{cond}	.F32	Sd, Sm	1	?	?	?	?	-	-	-	-	-
	Compare one register with zero, writes the result to the FPSCR flags.		V	CMPE	{cond}	.F32	Sd, #0	1	?	?	?	?	-	-	-	-	-
	Convert a floating-point value to a 32-bit signed/unsigned integer		V	CVT	{cond}	.F32.S32 .F32.U32	Sd, Sm	1	-	-	-	-	-	-	-	-	-
	Convert a floating-point value to a 16/32-bit signed/unsigned fixed-point. #bits is the number of fraction bots in the fixed-point number.		V	CVT	{cond}	.F32.S16 .F32.S32 .F32.U16 .F32.U32	Sd, Sd, #bits	1	-	-	-	-	-	-	-	-	-
	Convert a single-precision value to half-precision value. y=B: Sd _{15:0} ← SingleToHalf(Sm) y=T: Sd _{31:16} ← SingleToHalf(Sm)		V	CVTy y = B T	{cond}	.F32.F16	Sd, Sm	1	-	-	-	-	-	-	-	-	-
	Converts the value in a single-precision register to double-precision.		V	CVT	{cond}	.F32.F64	Sd, Dm	1	-	-	-	-	-	-	-	-	-
	Load Register Rn = Base register #imm = Immediate offset used to form the address	label = The label of the literal data item to be loaded	V	LDR	{cond}	.3264	Sd/Dd, [Rn {, #imm}] Sd/Dd, label Sd/Dd, [PC, #imm]	2 3	-	-	-	-	-	-	-	-	-
	Load Multiple Registers Rn = Base register, mode = IA (increment after) / DB (decrement before)	list = Registers to be loaded, consecutively numbered, separated by commas, surrounded by brackets	V	LDM	{mode} {cond}	.3264	Rn(!), list	1+N 1+2*N	-	-	-	-	-	-	-	-	-
	Store Single Register Sd/Dd = Source Register Rn = Base register	#imm = Immediate offset used to from the address	V	STR	{cond}	.3264	Sd/Dd, [Rn{, #imm}]	2 3	-	-	-	-	-	-	-	-	-
	Store Multiple Registers Rn = Base register, mode = IA (increment after) / DB (decrement before)	list = Registers to be loaded, consecutively numbered, separated by commas, surrounded by brackets	V	STM	{mode} {cond}	.3264	Rn(!), list	1+N 1+2*N	-	-	-	-	-	-	-	-	-
	Store multiple floating-point registers to stack list = Registers to be stored, list of consecutively numbered double- or single-precision floating-point registers separated by commas and surrounded by brackets.		V	PUSH	{cond}	.3264	list	1+N 1+2*N	-	-	-	-	-	-	-	-	-
	Load multiple floating-point registers to stack list = Registers to be loaded, list of consecutively numbered double- or single-precision floating-point registers separated by commas and surrounded by brackets.		V	POP	{cond}	.3264	list	1+N 1+2*N	-	-	-	-	-	-	-	-	-
	Copy the content of one register to another. Or places an immediate constant into the destination.	Sd/Dd ← Sm/Dm or Sd/Dd ← imm32/64	V	MOV	{cond}	.3264	Sd/Dd, Sm/Dm Sd/Dd, #imm	1	-	-	-	-	-	-	-	-	-
	Transfer one word from a register to the upper/lower half of a doubleword. Or vice versa. x is 1 for top half or 0 for bottom half.	Sd ← Rt or Rt ← Sn	V	MOV	{cond}	.32 or omitted	Ddx, Rt Rt, Dnx	1	-	-	-	-	-	-	-	-	-
	Transfer the contents of two single-precision registers to two registers, or the contents of two registers to a pair of single-precision registers.	Sm ← Rt Sm1 ← Rt2 or Rt ← Sm Rt2 ← Sm1	V	MOV	{cond}	-	Sm, Sm1, Rt, Rt2 Rt, Rt2, Sm, Sm1	2	-	-	-	-	-	-	-	-	-
	Transfer two words from two registers to a double-word register, or from a doubleword register to two registers.	Dm _{31:0} ← Rt Dm _{63:32} ← Rt2 or Rt ← Dm _{31:0} Rt2 ← Dm _{63:32}	V	MOV	{cond}	-	Dm, Rt, Rt2 Rt, Rt2, Dm	2	-	-	-	-	-	-	-	-	-
	Move the value of the FPSCR to a general-purpose register, or the FCSR flags to APSR.	if Rt = 15: APSR.N ← FPSCR.N APSR.Z ← FPSCR.Z APSR.C ← FPSCR.C APSR.V ← FPSCR.V else: Rt ← FPSCR	V	MRS	{cond}	-	Rt, FPSCR	1	?	?	?	?	-	-	-	-	-
	Move the value of a general-purpose register to FPSCR.	FPSCR ← Rt	V	MSR	{cond}	-	FPSCR, Rt	1	-	-	-	-	-	-	-	-	-

2 ZAHLENSYSTEME

2.1 Gleitkomma-Zahlen (IEEE-754)

$$W = (-1)^s \cdot (1.f)_2 \cdot 2^{e-Bias}$$

W Wert der reellen Zahl
s Vorzeichen Information (binär)
f Fraktion der Mantisse (binär)
e Charakteristik des Exponenten (binär)
Bias Exponenten-Versatz



Null	$W = (-1)^s \cdot 0$	$e = 0$	$f = 0$
Denormalisiert	$W = (-1)^s \cdot (0.f)_2 \cdot 2^{-126 \setminus -1022}$	$e = 0$	$f \neq 0$
Normalisiert	$W = (-1)^s \cdot (1.f)_2 \cdot 2^{e-127 \setminus -1023}$	$1 \leq e \leq 254 \setminus 2046$	$f = \text{beliebig}$
Unendlich	$W = (-1)^s \cdot \infty$	$e = 255 \setminus 2047$	$f = 0$
Nichtzahl	NaN	$e = 255 \setminus 2047$	$f \neq 0$

3 BIT-BANDING

3.1 Bit-Banding Memory Mapping

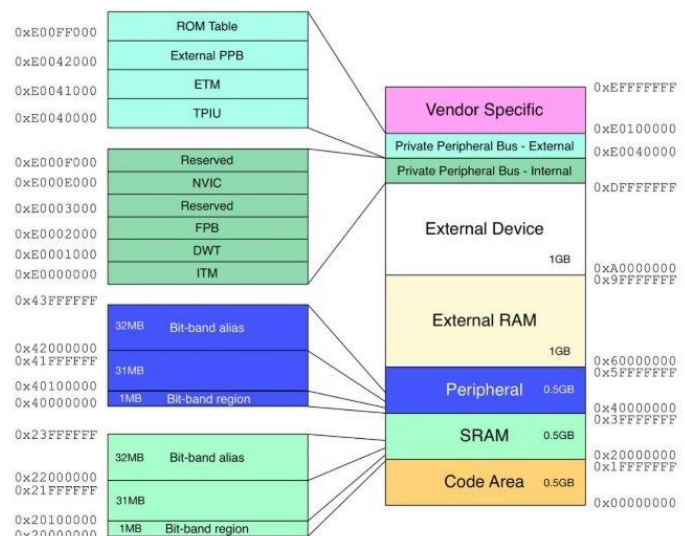
BBRB: Bit-Band Region Base
0x2000'0000 oder 0x4000'0000

BBRA: Bit-Band Region Adresse
0x2000'0000 - 0x2010'0000 oder
0x4000'0000 - 0x4010'0000

BBAB: Bit-Band Alias Base
0x2200'0000 oder 0x4200'0000

BBAA: Bit-Band Alias Adresse
0x2200'0000 - 0x23FF'FFFF oder
0x4200'0000 - 0x43FF'FFFF

BNr: Bit-Number, zu änderndes
Bit[0..31] an der BBRA



3.2 Berechnung der Bit-Band Alias Adresse (BBAA)

$$BBAA = BBAB + (BBRA - BBRB) \cdot 32 + 4 \cdot BNr$$

$$BBAA = BBAB + ((BBRA - BBRB) \ll 5) + (BNr \ll 2)$$

3.3 Berechnung der Bit-Band Region Adresse (BBRA) und der Bit-Nummer (BNr)

$$BBRA = BBRB + ((BBAA - BBAB) / (32 \cdot 4)) \cdot 4$$

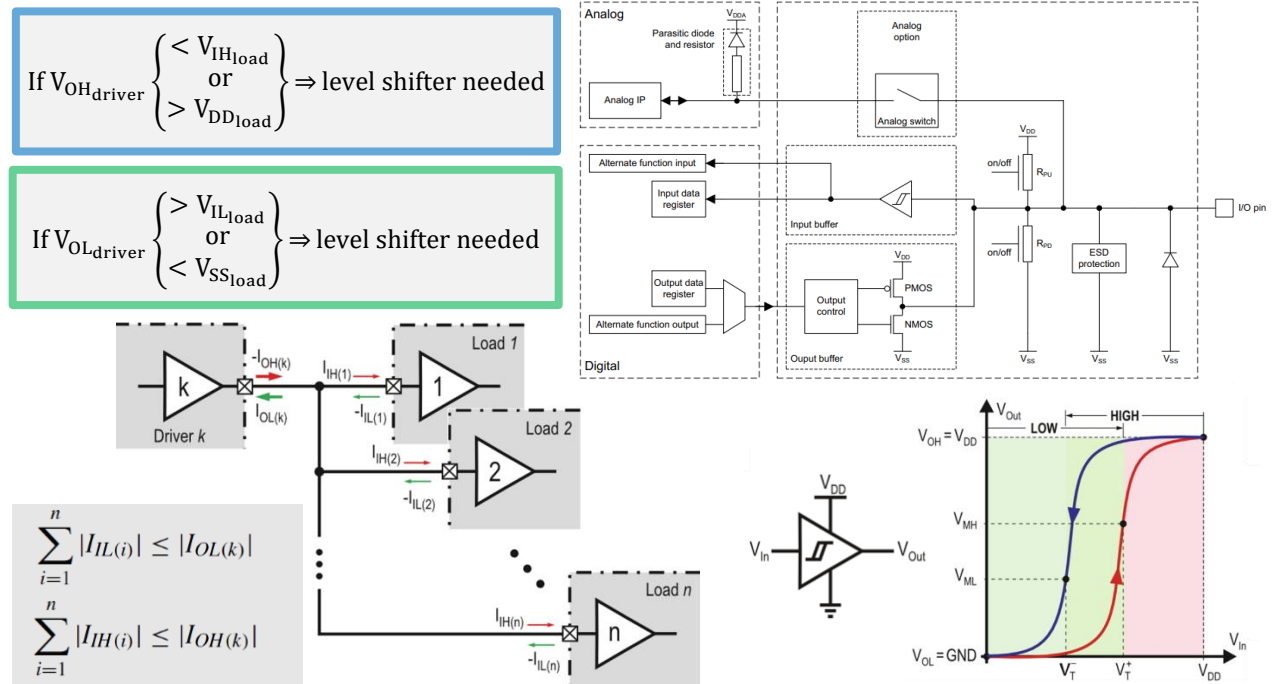
$$BBRA = BBRB + [((BBAA - BBAB) \gg 5) \& 0xFFFF'FFFC]$$

$$BNr = ((BBAA - BBAB) \% (32 \cdot 4)) / 4$$

$$BNr = ((BBAA - BBAB) \& 0x0000'007F) \gg 2$$

4 GPIO

4.1 GPIO Interface



4.2 GPIO Register

Offset	Name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x00	GPIOx_MODER	Mode15 [1:0]		Mode14 [1:0]		Mode13 [1:0]		Mode12 [1:0]		Mode11 [1:0]		Mode10 [1:0]		Mode9 [1:0]		Mode8 [1:0]		Mode7 [1:0]		Mode6 [1:0]		Mode5 [1:0]		Mode4 [1:0]		Mode3 [1:0]		Mode2 [1:0]		Mode1 [1:0]		Mode0 [1:0]	
	Reset Value	1	x	1	x	1	x	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	x	1	x	1	1	1	1	
0x04	GPIOx_OTYPER																	OT15	OT14	OT13	OT12	OT11	OT10	OT9	OT8	OT7	OT6	OT5	OT4	OT3	OT2	OT1	OT0
	Reset Value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0x08	GPIOx_OSPEEDR	SPEED15 [1:0]		SPEED14 [1:0]		SPEED13 [1:0]		SPEED12 [1:0]		SPEED11 [1:0]		SPEED10 [1:0]		SPEED9 [1:0]		SPEED8 [1:0]		SPEED7 [1:0]		SPEED6 [1:0]		SPEED5 [1:0]		SPEED4 [1:0]		SPEED3 [1:0]		SPEED2 [1:0]		SPEED1 [1:0]		SPEED0 [1:0]	
	Reset Value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0x0C	GPIOx_PUPDR	PUPD15 [1:0]		PUPD14 [1:0]		PUPD13 [1:0]		PUPD12 [1:0]		PUPD11 [1:0]		PUPD10 [1:0]		PUPD9 [1:0]		PUPD8 [1:0]		PUPD7 [1:0]		PUPD6 [1:0]		PUPD5 [1:0]		PUPD4 [1:0]		PUPD3 [1:0]		PUPD2 [1:0]		PUPD1 [1:0]		PUPD0 [1:0]	
	Reset Value	0	x	x	0	0	x	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	x	0	0	0	0	0	0	0	
0x10	GPIOx_IDR																	ID15	ID14	ID13	ID12	ID11	ID10	ID9	ID8	ID7	ID6	ID5	ID4	ID3	ID2	ID1	ID0
	Reset Value																	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	
0x14	GPIOx_ODR																	OD15	OD14	OD13	OD12	OD11	OD10	OD9	OD8	OD7	OD6	OD5	OD4	OD3	OD2	OD1	OD0
	Reset Value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

GPIOx_MODER: MODE[15:0][1:0]:
 00: Input mode
 10: Alternate Function
 01: Output mode
 11: Analog mode (reset state)

GPIOx_OTYPER: OT[15:0]:
 0: Output Push-Pull
 1: Output Open-Drain

GPIOx_OSPEEDR: OSPEED[15:0][1:0]:
 00: Low Speed
 10: High Speed
 01: Medium Speed
 11: Very High Speed

GPIOx_PUPDR: PUPD[15:0][1:0]:
 00: No Pull-Up/Down
 10: Pull-Down
 01: Pull-Up
 11: Reserve

GPIOx_IDR: ID[15:0]:
 Input Data Register. These bits are read-only.

GPIOx_ODR: OD[15:0]:
 Output Data Register. Read/written by software.

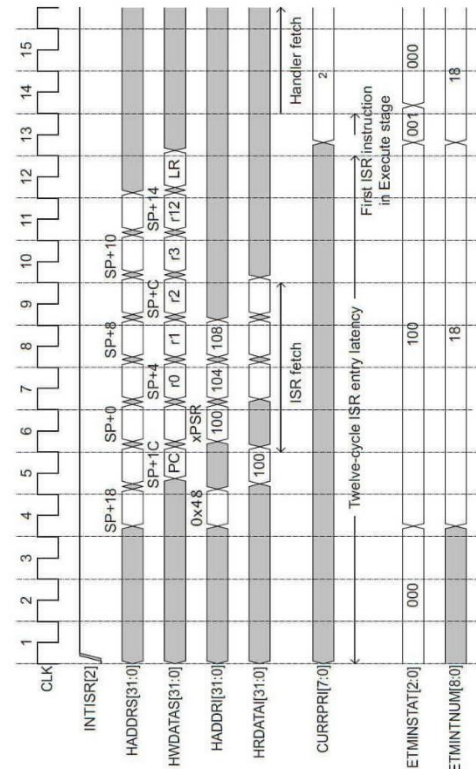
0x4800 1800 - 0x4800 1BFF	1 KB	GPIOG
0x4800 1400 - 0x4800 17FF	1 KB	GPIOF
0x4800 1000 - 0x4800 13FF	1 KB	GPIOE
0x4800 0C00 - 0x4800 0FFF	1 KB	GPIOD
0x4800 0800 - 0x4800 0BFF	1 KB	GPIOC
0x4800 0400 - 0x4800 07FF	1 KB	GPIOB
0x4800 0000 - 0x4800 03FF	1 KB	GPIOA



5 NVIC, INTERRUPTS & EXCEPTIONS

5.1 Vectortable, Vector-Fetch & Stacking

	Exception Number	Vectors	CMSIS Interrupt Number	Memory Address
Interrupts	255	Interrupt#239 Vector	239	0x0000'03FF
	...	...	...	...
	18	Interrupt#2 Vector	2	0x0000'0048
	17	Interrupt#1 Vector	1	0x0000'0044
	16	Interrupt#0 Vector	0	0x0000'0040
Exceptions	15	SysTick Vector	-1	0x0000'003C
	14	PendSV Vector	-2	0x0000'0038
	13	Not Used	NA	0x0000'0034
	12	Debug Monitor	NA	0x0000'0030
	11	SVC Vector	-5	0x0000'002C
	10	Not Used	NA	0x0000'0028
	9	Not Used	NA	0x0000'0024
	8	Not Used	NA	0x0000'0020
	7	Not Used	NA	0x0000'001C
	6	Usage Fault	NA	0x0000'0018
	5	Bus Fault	NA	0x0000'0014
	4	MemManageFault	NA	0x0000'0010
	3	Hard Fault Vector	-13	0x0000'000C
	2	NMI Vector	-14	0x0000'0008
	1	Reset Vector	NA	0x0000'0004
	0	MSP Initial Value	NA	0x0000'0000



5.2 xPSR Exception Number

	31	30	29	28	27	26:25	24	23:20	19:16	15:10	9	8	7	6	5	4:0
xPSR	N	Z	C	V	Q	ICI/IT	T			ICI/IT		Exception number				

5.3 NVIC: Spezielle Features

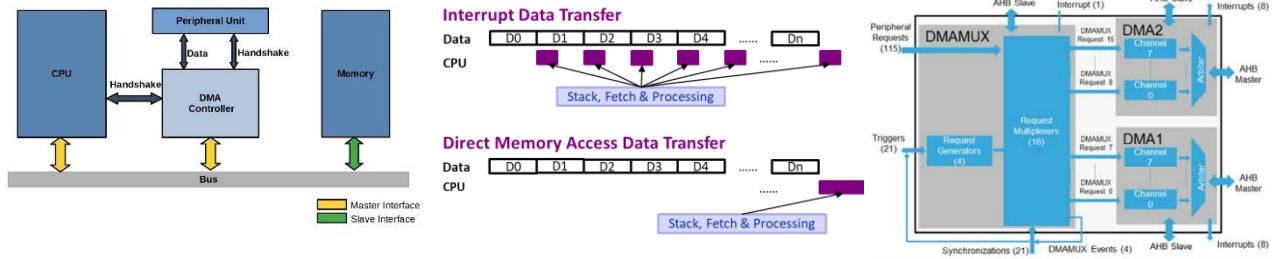
Tail Chaining: Ein tiefer priorisierter IRQ#2 wird am Ende eines aktiven IRQ#1 abgearbeitet. Dabei wird das Stacking übernommen und nur das Fetching aktualisiert (6 Cycles anstelle 24 Cycles).

Late Arrival: Ein höher priorisierter IRQ#2 kann während der Einsprunghase des IRQ#1 die Kontrolle übernehmen. Dabei wird das bereits angefangene Stacking übernommen und das Fetching aktualisiert (6-12 Cycles anstelle 12 Cycles).

POP-Preemption: Ein höher priorisierter IRQ#2 kann während der Rücksprungphase des IRQ#1 die Kontrolle übernehmen. Dabei wird das Stacking übernommen und das Fetching aktualisiert (6-12 Cycles anstelle 12 Cycles).

Lazy Stacking: Bei deaktivierter FPU werden nur die CPU-Register R0-R3, R12, LR, PC & xPSR auf dem Stack gesichert. Bei aktivierter FPU und Verwendung im Fore- & Background werden zusätzlich die FPU-Register S0-S15 & FPSCR auf dem Stack abgelegt. Dadurch vergrößert sich der Stackframe um 68Byte und die Latency auf 29 Cycles.

6 DMA



6.1 DMA Data Alignment

Byte/Halfword zu Word



Adresse	Data[7:0]
0xFFFF'XXX0	B0
0xFFFF'XXX1	B1
0xFFFF'XXX2	B2
0xFFFF'XXX4	B3

Adresse	Data[31:0]
0xFFFF'XXX0	0000'00B0
0xFFFF'XXX4	0000'00B1
0xFFFF'XXX8	0000'00B2
0xFFFF'XXXC	0000'00B3

Word zu Byte/Halfword



Adresse	Data[31:0]
0xFFFF'XXX0	B3B2'B1B0
0xFFFF'XXX4	B7B6'B5B4
0xFFFF'XXX8	BBA'B9B8
0xFFFF'XXxC	BFB'E'BD'BC

Adresse	Data[15:0]
0xFFFF'XXX0	B1B0
0xFFFF'XXX2	B5B4
0xFFFF'XXX4	B9B8
0xFFFF'XXX6	BDBC

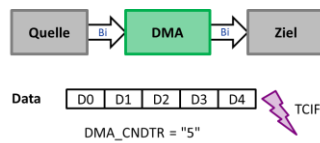
6.2 DMA Register

Register Name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DMA_ISR	TEIF8	HTIF8	TCIF8	GIF8	TEIF7	HTIF7	TCIF7	GIF7	TEIF6	HTIF6	TCIF6	GIF6	TEIF5	HTIF5	TCIF5	GIF5	TEIF4	HTIF4	TCIF4	GIF4	TEIF3	HTIF3	TCIF3	GIF3	TEIF2	HTIF2	TCIF2	GIF2	TEIF1	HTIF1	TCIF1	GIF1
Reset Value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
DMA_IFCR	CTEIF8	CHTIF8	CTCIF8	CGIF8	CTEIF7	CHTIF7	CTCIF7	CGIF7	CTEIF6	CHTIF6	CTCIF6	CGIF6	CTEIF5	CHTIF5	CTCIF5	CGIF5	CTEIF4	CHTIF4	CTCIF4	CGIF4	CTEIF3	CHTIF3	CTCIF3	CGIF3	CTEIF2	CHTIF2	CTCIF2	CGIF2	CTEIF1	CHTIF1	CTCIF1	CGIF1
Reset Value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
DMA_CCRx																		MEM2MEM				PL[1:0]					MSIZE[1:0]					
Reset Value																		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
DMA_CNDTRx																																
Reset Value																		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
DMA_CPARx	PA[31:0]																															
Reset Value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
DMA_CMARx	MA[31:0]																															
Reset Value	0	x	x	0	0	x	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	x	0	0	0	0	0	0	0	0

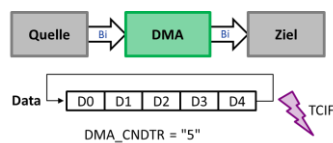
DMA_ISR:	GIFx TCIFx HTIFx TEIFx	(Global interrupt flag for channel x) (Transfer complete flag for channel x) (Half transfer flag for channel x) (Transfer error flag for channel x)	0: No TE, HT, or TC event 0: No TC event 0: No HT event 0: No TE event	1: TE, HT, or TC event occurred 1: TC event occurred 1: HT event occurred 1: TE event occurred
DMA_IFCR:	CGIFx CTCIFx CHTIFx CTEIFx	(Global interrupt flag clear for channel x) (Transfer complete flag clear for channel x) (Half transfer flag clear for channel x) (Transfer error flag clear for channel x)	0: Writing "0" has no effect 0: Writing "0" has no effect 0: Writing "0" has no effect 0: Writing "0" has no effect	1: Writing "1" clear corresponding flag 1: Writing "1" clear corresponding flag 1: Writing "1" clear corresponding flag 1: Writing "1" clear corresponding flag
DMA_CCRx:	EN TCIE HTIE TEIE DIR CIRC PINC MINC PSIZE[1:0] MSIZE[1:0] PL[1:0] MEM2MEM	(Channel enable) (Transfer complete interrupt enable) (Half transfer interrupt enable) (Transfer error interrupt enable) (Data transfer direction) (Circular mode) (Peripheral increment mode) (Memory increment mode) (Peripheral data size) (Memory data size) (Priority level) (Memory-to-memory mode)	0: Disabled (cleared by software) 0: Disabled (cleared by software) 0: Disabled (cleared by software) 0: Disabled (cleared by software) 0: Read from peripheral 0: Disabled (cleared by software) 0: Disabled (cleared by software) 0: Disabled (cleared by software) 0: Disabled (cleared by software) 0: Disabled (cleared by software) 0: Low 0: Disabled (cleared by software)	1: Enabled (set by software) 1: Enabled (set by software) 1: Enabled (set by software) 1: Enabled (set by software) 1: Read from memory 1: Enabled (set by software) 1: Enabled (set by software) 1: Enabled (set by software) 10: 32 bits 10: 32 bits 10: High 1: Enabled (set by software)
DMA_CNDTRx:	NDT[15:0]	(DMA channel x number of data to transfer)	Bits 15:0: Number of data to transfer (0 to 2 ¹⁶ - 1)	
DMA_CPARx:	PA[31:0]	(Peripheral/Memory address)	Bits 31:0: Contains the base address of the peripheral/memory data register	
DMA_CMARx:	MA[31:0]	(Memory/Peripheral address)	Bits 31:0: Contains the base address of the memory/peripheral data register	

6.3 DMA Transfer Mode

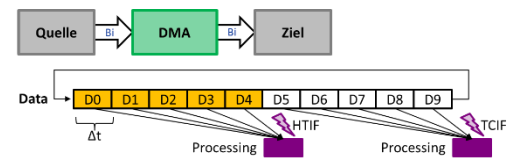
Normal Mode (CIR = "0")



Circular Mode (CIR = "1")

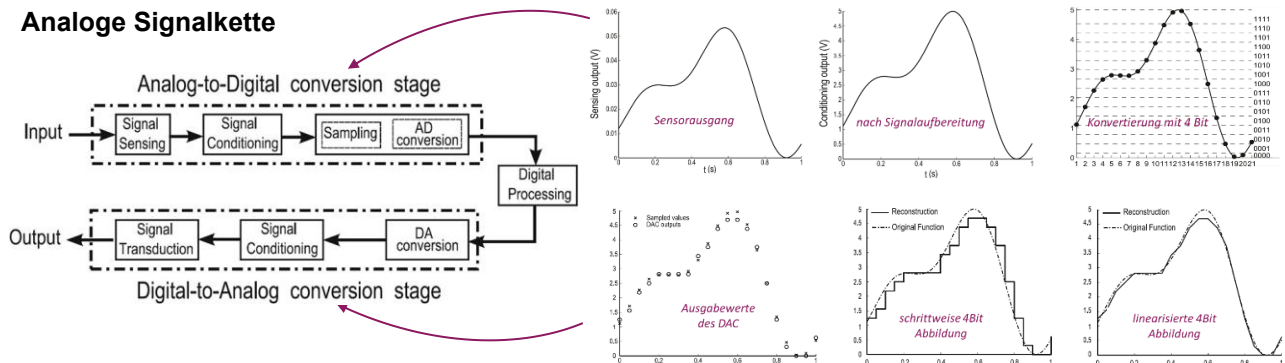


Circular Ping Pong Mode (CIR = "1")



7 ADC & DAC

Analoge Signalkette



Spannungsbereich:

$$V_{range} = V_{max} - V_{min}$$

Auflösung:

$$\Delta = \frac{V_{max} - V_{min}}{2^n} = \frac{V_{range}}{2^n}$$

Quantisierungsstufen:

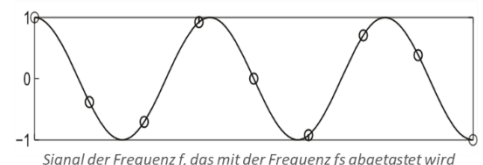
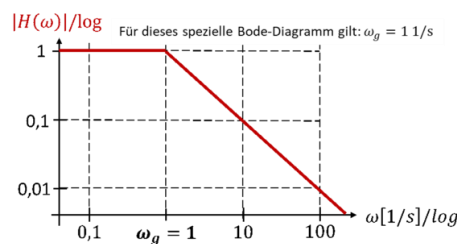
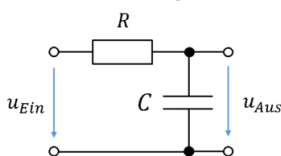
$$\{V_{min}, V_{min} + \Delta, V_{min} + 2\Delta, \dots, V_{min} + (2^n - 1)\Delta\}$$

Quantisierungsfehler:

$$\text{Quantisierungsfehler} = \text{Aktueller Wert} - \text{Quantisierungsstufe}$$

7.1 Abtastung und Aliasing

$$H(\omega) = \frac{u_{Aus}}{u_{Ein}} = \frac{1}{1 + j\omega/\omega_g}; \omega_g = \frac{1}{RC}$$



Nyquist-Theorem

Abtastfrequenz muss mind. doppelt so gross sein wie die höchste vorhandene Frequenzkomponente:

$$f_s \geq 2 \cdot f_H$$

f_s : Abtastfrequenz des Wandlers

f_H : Höchste Frequenzkomponente im Signal von Interesse

Aliasing

wenn ein Signal mit der Frequenz f mit einer Frequenz f_s abgetastet wird, gelten dieselben Samples für ein anderes Signal mit der Frequenz von $k \cdot f_s \pm f$ (siehe Abbildung links).

⇒ **Verwendung von Tiefpass- oder Bandpass-Antialiasing-Filtern ist nötig!**

7.2 Quantisierungsrauschdichte

Quantisierungsrauschdichte

$$v_{n(ADC)} = \frac{\Delta V}{\sqrt{12 \cdot f_s}} = \frac{V_{max} - V_{min}}{2^n \cdot \sqrt{12 \cdot f_s}}$$

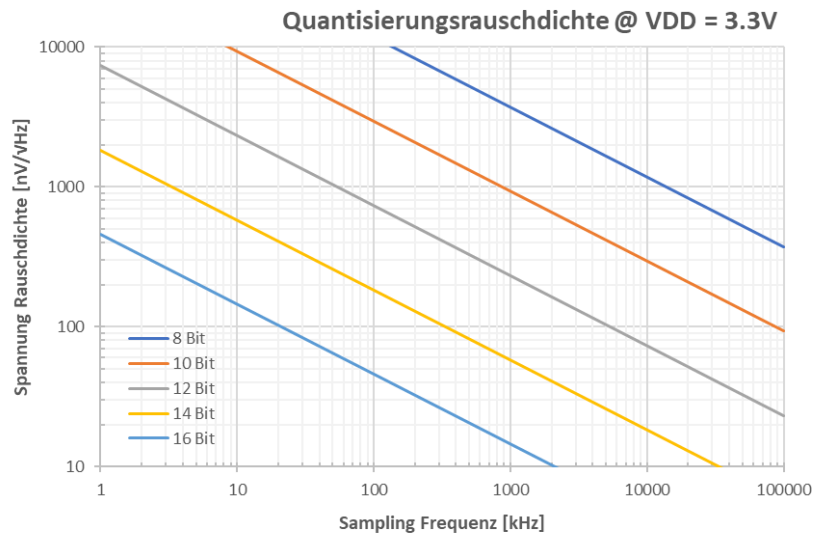
Dominante Systemrauschdichte

$$v_{n(ADC)} \leq \frac{v_{n(LNA)}}{2}$$

Minimale Samplingrate

$$f_s \geq \left(\frac{2 \cdot (V_{max} - V_{min})}{\sqrt{12} \cdot 2^n \cdot v_{n(LNA)}} \right)^2$$

$$= \frac{1}{3 \cdot 2^{2n}} \left(\frac{V_{max} - V_{min}}{v_{n(LNA)}} \right)^2$$



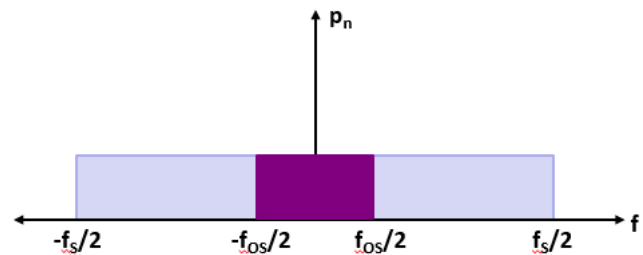
7.3 Oversampling

Leistungsrauschdichte

$$p_n = \frac{v_n^2}{1\Omega} = \frac{\Delta^2}{1\Omega \cdot 12 \cdot f_s}$$

Oversampling Faktor

$$M = \frac{f_s}{f_{OS}} = 4^K$$



Leistungsrauschen

$$p_{RMS} = \int_{-f_{OS}/2}^{f_{OS}/2} p_n \sigma f = p_n \cdot f_{OS} = \frac{\Delta^2 f_{OS}}{12 \cdot f_s} = \frac{\Delta^2}{12M} = \frac{\Delta^2}{12 \cdot 4^K}$$

Signal-Rausch-Verhältnis (dB)

$$SNR = 10 \cdot \log \left(\frac{p_n \cdot f_s}{p_{RMS}} \right) = 10 \cdot \log \left(\frac{\Delta V^2 \cdot f_s}{12 \cdot f_s} \cdot \frac{12M}{\Delta V^2} \right) = 10 \cdot \log M$$

Auflösung nach Oversampling

$$\text{Anzahl Bits} = \frac{SNR}{20 \cdot \log 2} + N = \frac{10 \cdot \log M}{10 \cdot \log(4)} + N = \log_4 M + N = \log_4 4^K + N = K + N$$

⇒ **HW-Oversampling mit integrierter digitalen Summenbildung inkl. Shift (Mittelwertfilter) oder SW-Oversampling mittels implementiertem FIR- oder IIR-Filter.**

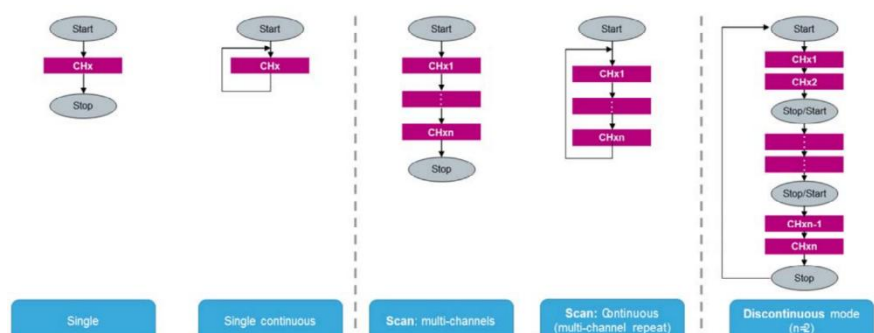
7.4 ADC-Sequenzen

Sequenzielle Abarbeitung

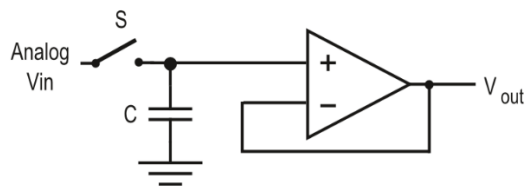
Wandlung von mehreren Channels pro ADC

Sample & Hold individuell konfigurierbar

- 18 frei definierbare Channels pro ADC
- Frei programmierbare Sample & Hold Time



7.5 Sample & Hold



Maximaler Eingangswiderstand

- Sample&Hold-Switch ist berücksichtigt
- Alle Widerstände (Parallel und in Serie), welche am Eingang angeschlossen sind

Maximaler Eingangswiderstand					
Cycles @ 60MHz	Sampling Time [ns]	R _{AIN(max)} [Ω] @ 12Bit	R _{AIN(max)} [Ω] @ 10Bit	R _{AIN(max)} [Ω] @ 8Bit	R _{AIN(max)} [Ω] @ 6Bit
2.5	41.7	100	120	180	220
6.5	108	330	390	470	560
12.5	208	680	820	1000	1200
24.5	408	1500	1500	1800	2700
47.5	792	2200	2200	2700	3900
92.5	1542	4700	5600	6800	8200
247.5	4125	12000	12000	15000	18000
640.5	10675	39000	47000	50000	50000

8 RESET & CLOCK CONTROL (RCC)

8.1 Power-On Reset (POR)

Eigenschaften:

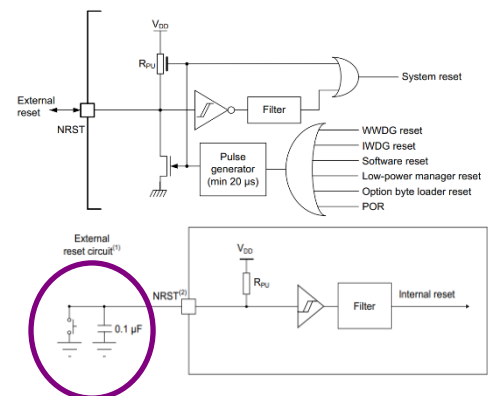
- Open-Drain-Schaltung mit Pull-Up Widerstand
- Reset kann extern ausgelöst werden
- Diverse interne Reset-Quellen

Verwendungszweck

- Verhindern von unerwünschten Zuständen
- Sicherstellung der korrekten Initialisierung
- Fehlererkennung & zeitliche Steuerung

Externe Beschaltung

- Taster für manuelle Auslösung
- Filterkapazität für mehr Störimmunität



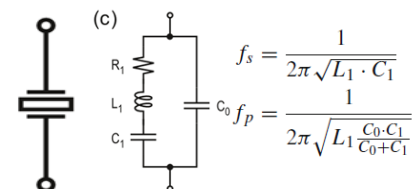
8.2 Quartz Resonator

Basiert auf mechanischer Resonanz eines schwingenden piezoelektrischen Kristalls

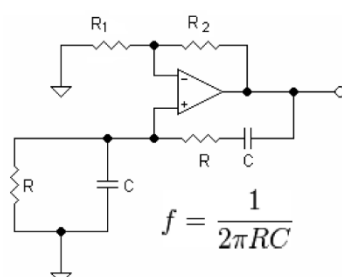
- Form und Größe des Kristalls bestimmen die Frequenz
- Verhält sich bei Erregung wie ein RLC-Resonanzkreis

Bei Erregung wird ein Signal mit sehr präziser Frequenz erzeugt

- Hervorragende Frequenzstabilität (Temperatur, Alterung, etc.)
- Hoher Wirkungsgrad (hoher Q-Faktor >10'000)
- Extrem breites Spektrum an Taktfrequenzen (10kHz...500MHz)
- **Problem:** da $C_0 \gg C_1$ liegen f_s und f_p sehr nahe zusammen

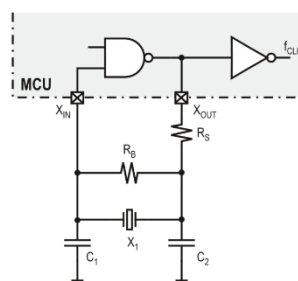


8.3 Oszillatoren



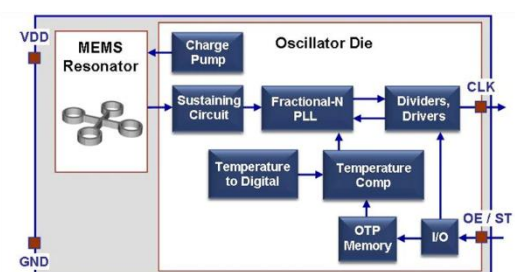
RC-basierter Oszillator

- Hohe absolute Abweichung
- Temperaturabhängiger Drift
- Alterungsabhängiger Drift
- Einfach und stromsparend
- Kleine Güte, schnelles Einschwingen



Quartz-basierter Oszillator

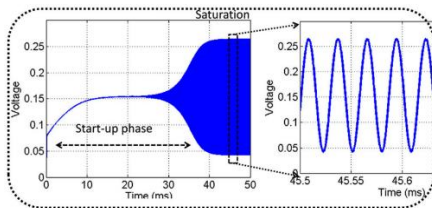
- Sehr kleine Toleranzen
- Kleine Driftabhängigkeiten
- Hoher Stromverbrauch
- Resonator bleibt extern
- Hohe Güte, langsames Einschwingen



MEMS-basierter Oszillator

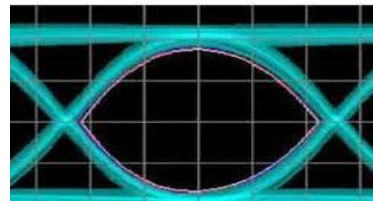
- Sehr kleine Toleranzen
- Kleine Driftabhängigkeiten
- Mittlerer Stromverbrauch
- Resonator und Oszillator bleiben extern (Kosten)
- Hohe Güte, gutes Einschwingen

8.4 Einschwingen & Stabilität



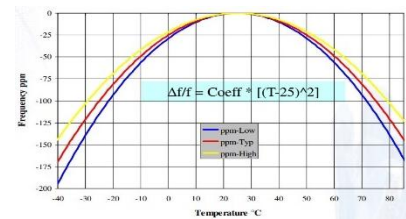
Einschwingen

- Je höher die Güte Q, umso länger dauert das Einschwingen.
- AGC und Start-Up-Schaltungen verbessern die Einschwingzeit.
- Einschwingzeit ist konfigurierbar



Kurzfristige Instabilität

- Jitter oder Phasenrauschen.
- Statistische Änderung über ein kurzes Zeitintervall (Periode)
- Werden häufig in «%» oder «‰» angegeben.



Langfristige Instabilität

- Drift (meistens Temperatur).
- Statistische Änderung über ein langes Zeitintervall
- Werden häufig in «ppm» oder «ppm/°C» angegeben.

8.5 Phase Locked Loop (PLL)

PLLM-Divider

- Digitaler Frequenzteiler (/1, /2, /3..., /16)

Frequenzmultiplikation N

- Frequenzmultiplikator (8x, ..., 127x)

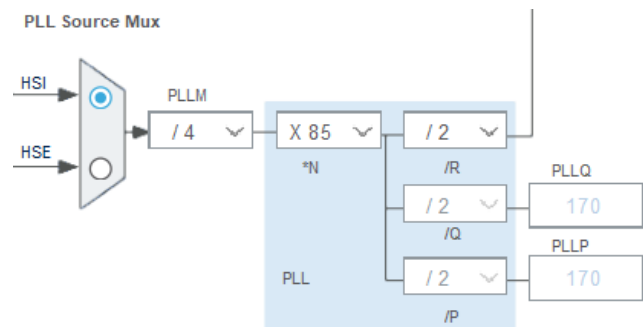
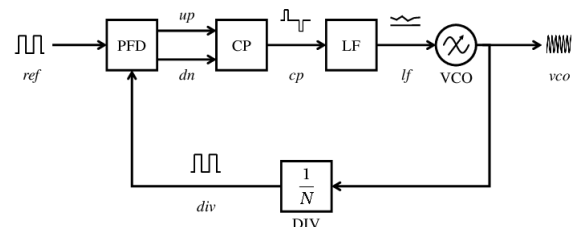
R-, Q-, & P-Divider

- Digitaler Frequenzteiler (/2, /4, /6, /8)

Berechnung der PLL-Ausgangsfrequenz

- $PLLCLK = \frac{HSI \cdot N}{PLLM \cdot R}$ oder $PLLCLK = \frac{HSE \cdot N}{PLLM \cdot R}$
- $PLLQ = \frac{HSI \cdot N}{PLLM \cdot Q}$ oder $PLLQ = \frac{HSE \cdot N}{PLLM \cdot Q}$
- $PLL P = \frac{HSI \cdot N}{PLLM \cdot P}$ oder $PLL P = \frac{HSE \cdot N}{PLLM \cdot P}$

Max. Systemfrequenz: 170MHz!



9 BASIS TIMER

Control-Register (CR)

Einstellungen für Counter (MUX, Count-Mode, etc.)

Prescaler-Register (PSCR)

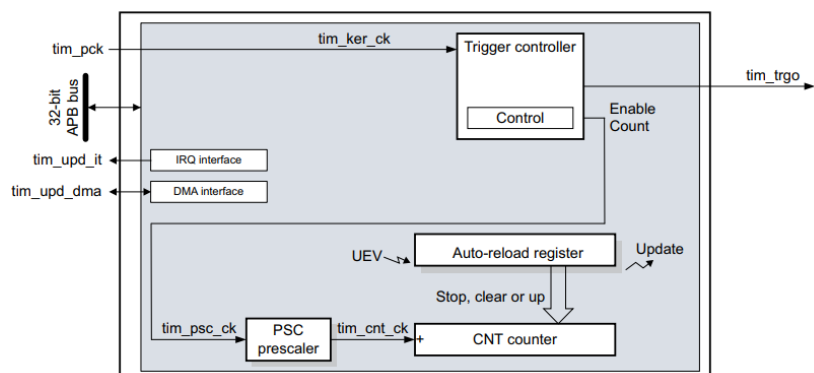
Vorteiler des Timer-Clocks

Counter-Register (CNTR)

Binäre Zählerbasis

Auto-Reload-Register (ARR)

Reload-Wert des Counters mit Update-Ereignis (Overflow)



Edge-Aligned Upcounting Mode



Edge-Aligned Downcounting Mode



Center-Aligned (Up-Down) Mode



10 SERIELLE SCHNITTSTELLEN

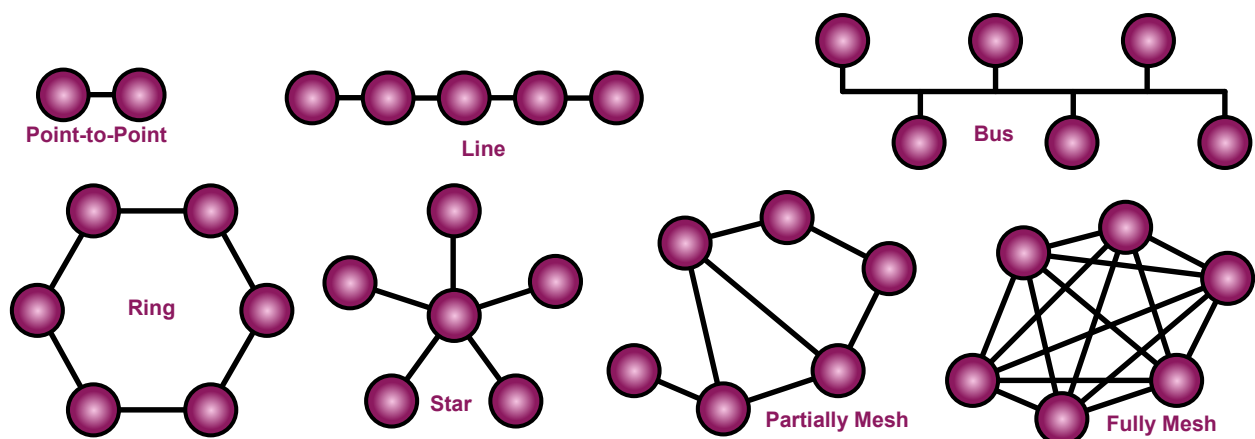
10.1 ASCII-Tabelle

ASCII 7Bit												Extended ASCII 8Bit											
Dez	Hex	Symbol	Dez	Hex	Symbol	Dez	Hex	Symbol	Dez	Hex	Symbol	Dez	Hex	Symbol	Dez	Hex	Symbol	Dez	Hex	Symbol	Dez	Hex	Symbol
0	0x00	NULL	32	0x20		64	0x40	@	96	0x60	`	128	0x80	€	160	0xA0		192	0xC0	À	224	0xE0	à
1	0x01	SOH	33	0x21	!	65	0x41	A	97	0x61	a	129	0x81		161	0xA1	í	193	0xC1	Á	225	0xE1	á
2	0x02	STX	34	0x22	"	66	0x42	B	98	0x62	b	130	0x82	,	162	0xA2	ç	194	0xC2	Â	226	0xE2	â
3	0x03	ETX	35	0x23	#	67	0x43	C	99	0x63	c	131	0x83	f	163	0xA3	£	195	0xC3	Ã	227	0xE3	ã
4	0x04	EOT	36	0x24	\$	68	0x44	D	100	0x64	d	132	0x84	"	164	0xA4	¤	196	0xC4	Ä	228	0xE4	ä
5	0x05	ENQ	37	0x25	%	69	0x45	E	101	0x65	e	133	0x85	...	165	0xA5	¥	197	0xC5	Å	229	0xE5	å
6	0x06	ACK	38	0x26	&	70	0x46	F	102	0x66	f	134	0x86	†	166	0xA6	¦	198	0xC6	Æ	230	0xE6	æ
7	0x07	BEL	39	0x27	'	71	0x47	G	103	0x67	g	135	0x87	‡	167	0xA7	§	199	0xC7	Ç	231	0xE7	ç
8	0x08	BS	40	0x28	(	72	0x48	H	104	0x68	h	136	0x88	ˆ	168	0xA8	¨	200	0xC8	È	232	0xE8	è
9	0x09	HT	41	0x29	)	73	0x49	I	105	0x69	i	137	0x89	%	169	0xA9	©	201	0xC9	É	233	0xE9	é
10	0x0A	LF	42	0x2A	*	74	0x4A	J	106	0x6A	j	138	0x8A	Š	170	0xAA	ª	202	0xCA	Ê	234	0xEA	ê
11	0x0B	VT	43	0x2B	+	75	0x4B	K	107	0x6B	k	139	0x8B	‹	171	0xAB	«	203	0xCB	Ë	235	0xEB	ë
12	0x0C	FF	44	0x2C	,	76	0x4C	L	108	0x6C	l	140	0x8C	Œ	172	0xAC	¬	204	0xCC	Ì	236	0xEC	ì
13	0x0D	CR	45	0x2D	-	77	0x4D	M	109	0x6D	m	141	0x8D	ˆ	173	0xAD	ˆ	205	0xCD	Í	237	0xED	í
14	0x0E	SO	46	0x2E	.	78	0x4E	N	110	0x6E	n	142	0x8E	Ž	174	0xAE	®	206	0xCE	Î	238	0xEE	î
15	0x0F	SI	47	0x2F	/	79	0x4F	O	111	0x6F	o	143	0x8F		175	0xAF		207	0xCF	Ï	239	0xEF	ï
16	0x10	DLE	48	0x30	0	80	0x50	P	112	0x70	p	144	0x90		176	0xB0	°	208	0xD0	Ð	240	0xF0	ð
17	0x11	DC1	49	0x31	1	81	0x51	Q	113	0x71	q	145	0x91	‘	177	0xB1	±	209	0xD1	Ñ	241	0xF1	ñ
18	0x12	DC2	50	0x32	2	82	0x52	R	114	0x72	r	146	0x92	’	178	0xB2	²	210	0xD2	Ò	242	0xF2	ò
19	0x13	DC3	51	0x33	3	83	0x53	S	115	0x73	s	147	0x93	“	179	0xB3	³	211	0xD3	Ó	243	0xF3	ó
20	0x14	DC4	52	0x34	4	84	0x54	T	116	0x74	t	148	0x94	”	180	0xB4	´	212	0xD4	Ô	244	0xF4	ô
21	0x15	NAK	53	0x35	5	85	0x55	U	117	0x75	u	149	0x95	•	181	0xB5	µ	213	0xD5	Õ	245	0xF5	õ
22	0x16	SYN	54	0x36	6	86	0x56	V	118	0x76	v	150	0x96	–	182	0xB6	¶	214	0xD6	Ö	246	0xF6	ö
23	0x17	ETB	55	0x37	7	87	0x57	W	119	0x77	w	151	0x97	—	183	0xB7	·	215	0xD7	×	247	0xF7	×
24	0x18	CAN	56	0x38	8	88	0x58	X	120	0x78	x	152	0x98	ˆ	184	0xB8	ˆ	216	0xD8	Ø	248	0xF8	ø
25	0x19	EM	57	0x39	9	89	0x59	Y	121	0x79	y	153	0x99	™	185	0xB9	™	217	0xD9	Ù	249	0xF9	ù
26	0x1A	SUB	58	0x3A	:	90	0x5A	Z	122	0x7A	z	154	0x9A	š	186	0xBA	°	218	0xDA	Ú	250	0xFA	ú
27	0x1B	ESC	59	0x3B	;	91	0x5B	[	123	0x7B	{	155	0x9B	›	187	0xBB	»	219	0xDB	Û	251	0xFB	û
28	0x1C	FS	60	0x3C	<	92	0x5C	\	124	0x7C		156	0x9C	œ	188	0xBC	¼	220	0xDC	Ü	252	0xFC	ü
29	0x1D	GS	61	0x3D	=	93	0x5D	]	125	0x7D	}	157	0x9D	ˆ	189	0xBD	½	221	0xDD	Ý	253	0xFD	ý
30	0x1E	RS	62	0x3E	>	94	0x5E	^	126	0x7E	~	158	0x9E	ž	190	0xBE	¾	222	0xDE	Þ	254	0xFE	þ
31	0x1F	US	63	0x3F	?	95	0x5F	_	127	0x7F	DEL	159	0x9F	Ÿ	191	0xBF	¿	223	0xDF	ß	255	0xFF	ÿ

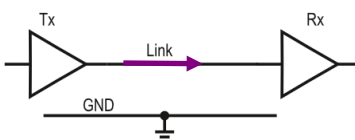
10.2 Allgemeine Berechnungen

Baudrate: $f_{\text{Baud}} = \frac{1}{t_{\text{bit}}}$ Bitrate: $f_{\text{Bit}} = n \cdot f_{\text{Baud}} = \frac{n}{t_{\text{bit}}}$ Nutzdatenrate: $f_{\text{Nutz}} = \frac{\text{Nutzdatenlänge}}{\text{Framelänge}} \cdot f_{\text{Baud}} = \frac{n \cdot \text{Nutzdatenlänge}}{\text{Framelänge} \cdot t_{\text{Bit}}}$

10.3 Verbindungstopologien



10.4 Single Ended vs. Differential Link

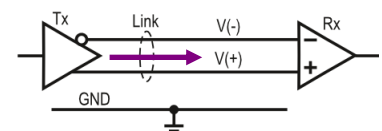


Single Ended Link

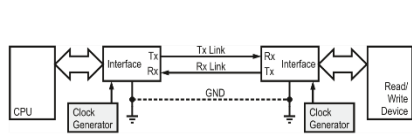
- Datenleitung ist auf GND-Potential referenziert
- Min. 2 Leitungen nötig (Link, GND)

Differential Link

- Zwei Datenleitungen mit invertierten Signalen
- Robust gegenüber äusseren Störeinflüssen
- Min. 3 Leitungen nötig (V+, V-, GND)



10.5 Asynchrone vs. Synchrone Datenübertragung

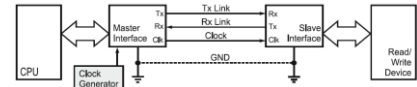


Asynchrone Übertragung

- keine zusätzliche Leitung für ein Clock-Signal
- Clocks müssen synchronisiert werden

Synchrone Übertragung

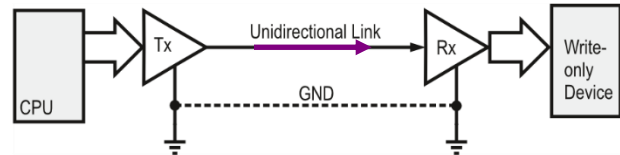
- Benötigt zusätzliche Leitung für ein Clock-Signal
- Schnelle und zuverlässige Datenübertragung
- Kein Synchronisationsheader & -Footer



10.6 Ausführung des Datenkanals

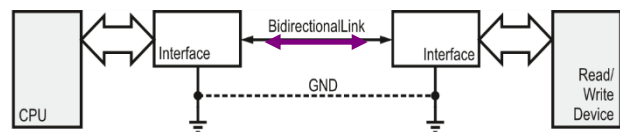
Simplex

- Unidirektionale Verbindung zwischen zwei Teilnehmern
- Überträgt permanent nur in eine Richtung
- Kein Bestätigen des Datenempfangs möglich



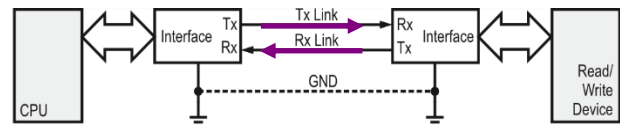
Half Duplex

- Bidirektionale Verbindung zwischen zwei Teilnehmern
- Ermöglicht sequenzielle Kommunikation in beide Richtungen
- Serielle Transceiver auf beiden Seiten erforderlich
- Verwendet in der Regel ein Kommunikationsprotokoll



Full Duplex

- Zwei separate unidirektionale Verbindungen
- Transmitter/Receiver auf beiden Seiten erforderlich
- Gleichzeitige Kommunikation in beide Richtungen



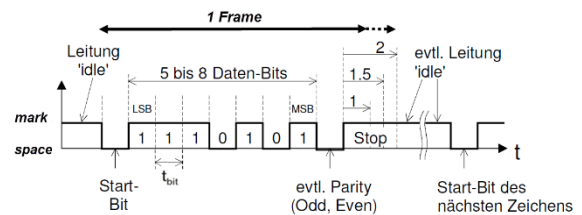
11 UNIVERSAL ASYNCHRONOUS RECEIVE/TRANSMIT (UART)

Baudrate: 75, 150, 300, 600, 1'200, 2'400, 4'800, 9'600, 19'200, 38'400, 57'600, 115'200

Header: 1 Start-Bit: immer logisch "0" oder space

Body: 5 - 8 Daten-Bits (LSB-first)

Footer: 1 Parity-Bit Even, Odd oder None (E,O,N)
1/1.5/2 Stop-Bits: immer logisch "1" oder mark

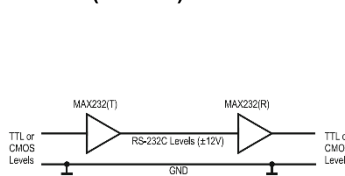


Absolute Taktabweichung:
$$\frac{\Delta t_{max}}{1Bit} = \pm \frac{t_{Bit}}{2 \cdot Framelänge} = \pm \frac{1Bit}{2 \cdot Baudrate \cdot Framelänge} \left[\frac{s}{Bit} \right]$$

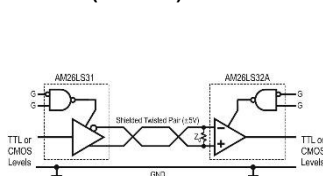
Relative Taktabweichung:
$$\frac{\Delta t_{max}}{t_{Bit}} = \pm \frac{t_{Bit} \cdot 1Bit}{2 \cdot Framelänge \cdot t_{Bit}} = \pm \frac{1Bit}{2 \cdot Framelänge} [\%]$$

11.1 Erweiterte Schnittstellen

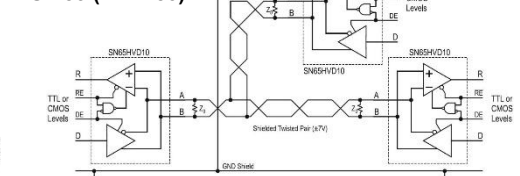
RS-232 (EIA-232)



RS-422 (EIA-422)



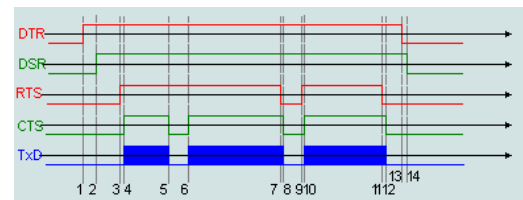
RS-485 (EIA-485)



11.2 Datenfluss-Steuerung

Die Flusskontrolle mit Hilfe von RS-232-Leitungen

- **RTS:** Request to Send
- **CTS:** Clear to send
- **DSR:** Data Set ready
- **DTR:** Data Terminal ready



12 SERIAL PERIPHERAL INTERFACE (SPI)

Synchroner serieller Bus-Standard

Full-Duplex Fähigkeit (bedingt)

Single-Ended TTL-Signalpegel (3.3V/5.0V)

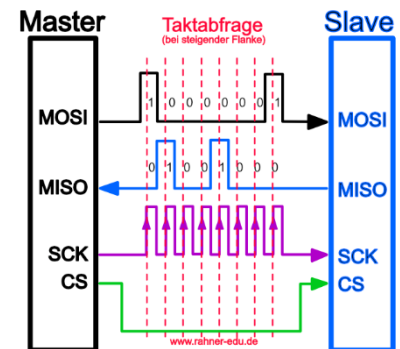
unterstützt die Kommunikation zwischen einem Main-Prozessor und einer oder mehreren Sub-Peripherien

Spezifikationen (Motorola 1987) definiert nur den physikalischen Layer

Paketweiterleitung zwischen Main und mehreren Subs

spezifiziert kein Daten-, Sitzungs- oder höhere Protokolle

Schieberegister, welches als Empfänger und Sender dient und durch den Übertragungstakt synchronisiert wird



12.1 Protokoll

Einstellungen

- Taktfrequenz frei wählbar (Main)
- Main muss Sub-Spezifikationen einhalten
- 4 unterschiedliche Clock-Modes möglich

Clock Polarity (CPOL)

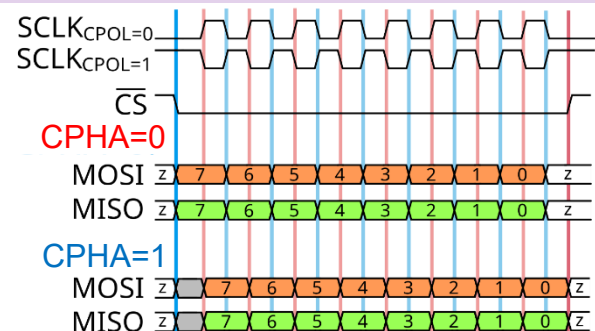
- 0: SCLK-Idle ist auf logisch "0"
- 1: SCLK-Idle ist auf logisch "1"

Clock Phase (CPHA)

- 0: Bit wird bei erster SCK-Flanke gelesen
- 1: Bit wird bei zweiter SCK-Flanke gelesen

Sub Select (SS)

- Auswahl des zu adressierenden Subs
- Signalisiert Start & Ende des Datentransfers
- Enable-Signal für die Schieberegister

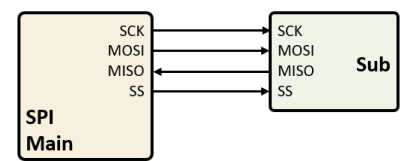
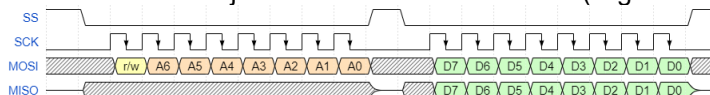


Mode	CPOL	CPHA
0	0	0
1	0	1
2	1	0
3	1	1

12.2 Single-Main / Single-Sub

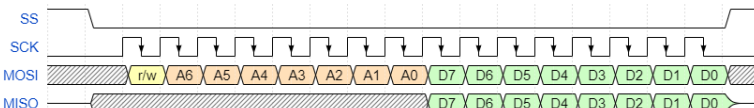
Variante 1

- SS wird nach jedem Frame kurz deaktiviert (allgemein gültig)



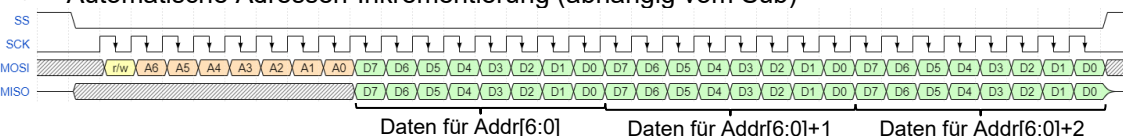
Variante 2

- SS wird erst nach einem Adressen/Daten-Paket deaktiviert (abhängig vom Sub)



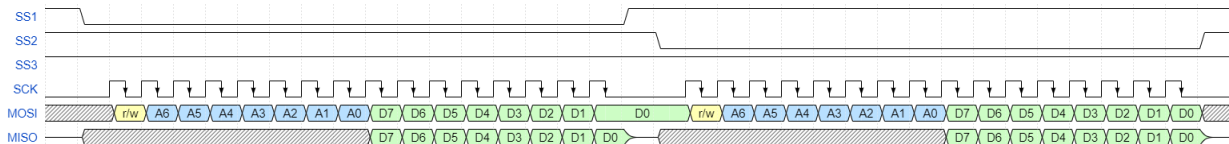
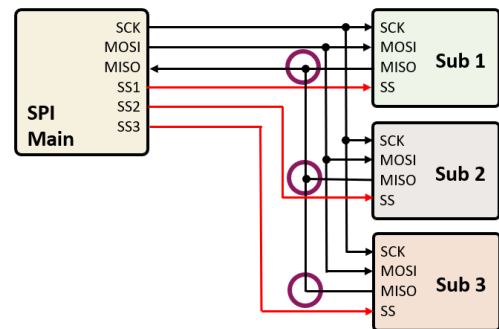
Variante 3

- Automatische Adressen-Inkrementierung (abhängig vom Sub)



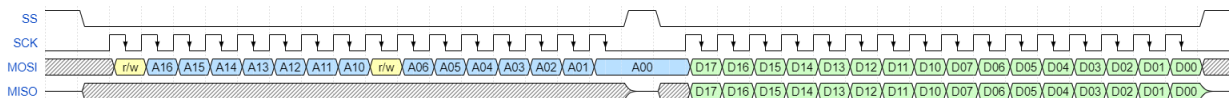
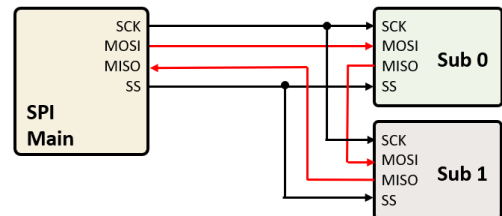
12.3 Single-Main / Multiple-Subs

- Stern- oder genauer Multi-Drop-Topologie
- Adressierung erfolgt über separate SS-Signale
- Gemeinsam genutzte MISO-Leitung
- Jeder Sub muss einen MISO-Tristate-Driver besitzen
- MISO-Treiber ist nur bei entsprechendem SS-Signal aktiv
- Gleiche Übertragung wie Single-Main / Single-Sub



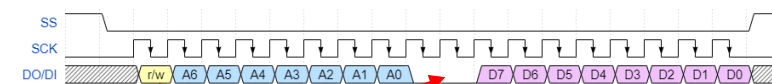
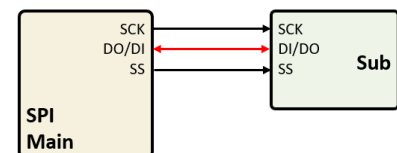
12.4 Single-Main / Chained-Subs

- Alle Subs teilen sich die gleiche SS-Leitung
- Die MISO/MOSI-Leitung wird durchgeschleift
- SS muss nach Adressübergabe deaktiviert werden
- Vergrößerung der Registerbreite auf n*Framelänge
- Main muss die Anzahl der Subs kennen
- Bei allen Subs muss das r/w-Bit gleichgesetzt werden



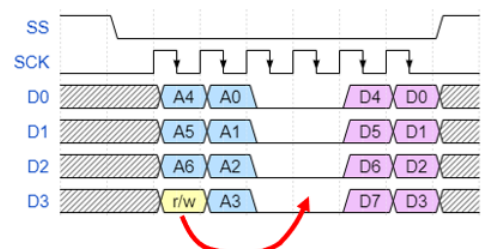
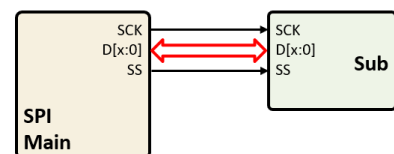
12.5 3-Wire Interface

- Nur drei Leitungen nötig (Half-Duplex Betrieb)
- Kleinere Übertragungsraten wegen "Bus Direction Swap"
- MISO und MOSI werden zusammengeführt



12.6 -MultiBit Interface (QSPI)

- Ähnlich 3-Wire Half-Duplex-Betrieb
- Erhöhung der Übertragungsrate
- Häufig eingesetzt bei schnellen Peripherie-Bausteinen
- **Dual-SPI**: Daten-Bits werden paarweise übertragen
- **Quad-SPI**: Daten-Bits werden als Nibble übertragen
- **Octal-SPI**: Daten-Bits werden als Byte übertragen



13 INTER-INTEGRATED CIRCUIT BUS (I²C-BUS)

SDA: Serial Data

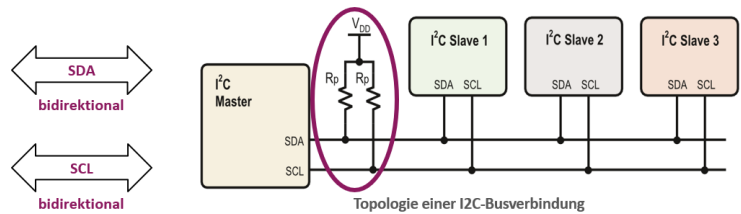
- führt die zu übertragenden Daten
- Open-Drain (Collector) Treiber

SCL: Serial Clock

- Synchronisiert den Bus-Transfers
- Open-Drain (Collector) Treiber

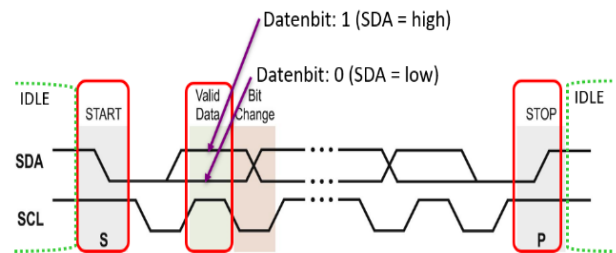
GND: Ground

- Referenzsignal für SDA und SCL

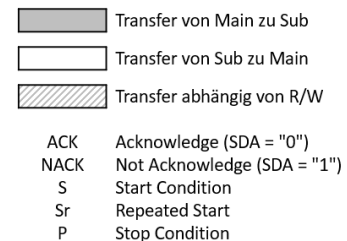
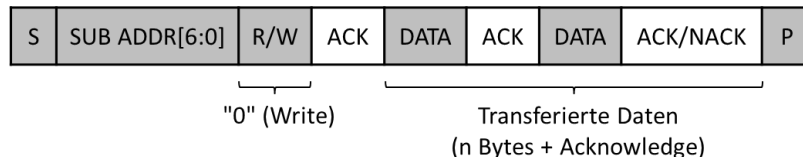


13.1 Protokoll

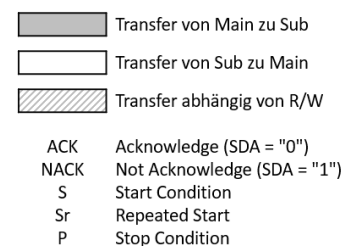
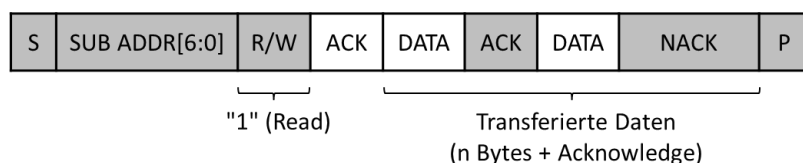
- **Start oder Restart Bedingung (S oder SR)**
 - Main übernimmt Kontrolle und initialisiert einen Transfer
 - Kann auch als Re-Start innerhalb eines Transfers genutzt werden
- **Daten-Transfer**
 - SCL taktet mit "Low-High-Low" Sequenz
 - SDA darf nur ändern wenn SCL = "Low"
- **Stop Bedingung (P)**
 - Schliesst einen Transfer ab
- **Idle Status**
 - SDA und SCL = "High"



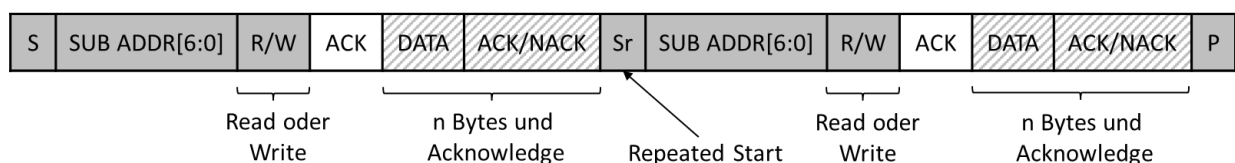
13.2 Schreibzugriff



13.3 Lesezugriff



13.4 Schreib- & Lesezugriff



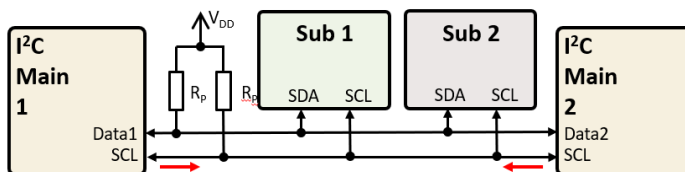
13.5 Spezielle Adressen

7 Bit Adressierung

- Theoretisch bis zu 128 Teilnehmer am I2C-Bus möglich
- Zwei Blöcke von je 8 Adressen sind reserviert
- effektiv sind 112 Teilnehmer möglich

Adresse + R/W	Funktion
0b0000'0000	General Call (Adressiert alle Teilnehmer)
0b0000'0001	Start Byte (Zur Aktivierung von Teilnehmern)
0b0000'001X	CBUS-Format (spezielles 3-Wire Bus Format)
0b0000'010X	Reserviert (für spezielle Bus-Formate)
0b0000'011X	Reserviert (für zukünftige Zwecke)
0b0000'1XXX	High-Speed Master Code
0b1111'0XXX	10Bit Adressierung
0b1111'1XXX	Reserviert (für zukünftige Zwecke)

13.6 Arbitrierung & Clock Stretching



Open-Drain (Collector) Treiber

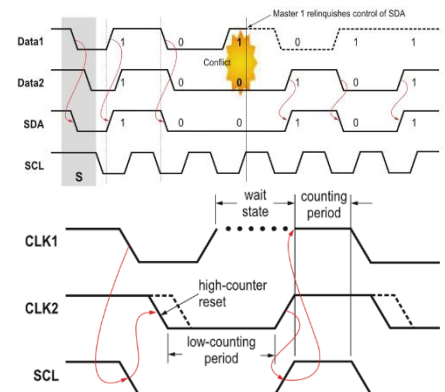
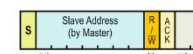
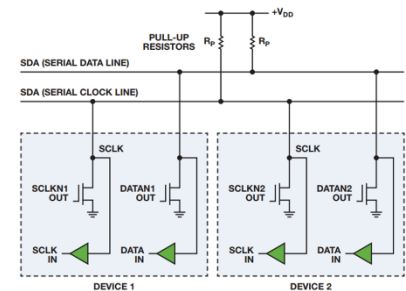
- Oder-Verknüpfung von Ausgängen (keine Kurzschlüsse)
- "Low"-State dominiert über "High"-State
- Ist auf beiden Leitungen vorhanden (SDA und SCL)

Arbitrierung

- Derjenige Main, welcher einen High-Pegel platzieren will, verliert die Kontrolle über den I2C-Bus
- Der Main, welcher den Sub mit der niedrigsten Adresse anspricht, gewinnt die Arbitrierung

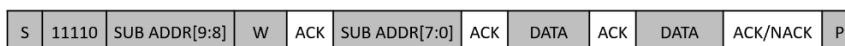
Clock Stretching

- Erlaubt Synchronisation bei Multi-Main und Datenbussteuerung bei Subs
- Jeder Busteilnehmer kann mit einem "Low"-Pegel auf dem SCL den Transfer blockieren

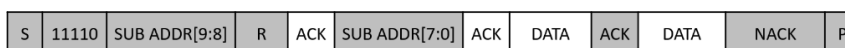


13.7 10Bit Adressierung

Schreibzugriff



Lesezugriff



Transfer von Main zu Sub

Transfer von Sub zu Main

ACK Acknowledge (SDA = "0")
NACK Not Acknowledge (SDA = "1")
S Start Condition
Sr Repeated Start Condition
P Stop Condition

13.8 High Speed Mode

Schreibzugriff



Lesezugriff



Transfer von Main zu Sub

Transfer von Sub zu Main

ACK Acknowledge (SDA = "0")
NACK Not Acknowledge (SDA = "1")
S Start Condition
Sr Repeated Start Condition
P Stop Condition

PERSONAL NOTES

PERSONAL NOTES